

软件工程基础

第1章：软件与软件工程

杨溢龙

北航软件学院

目　录

Contents

软件工程基础

第1章：软件与软件工程



Part 1 软件



Part 2 软件发展与软件危机



Part 3 软件工程

北京航空航天大學
BEIHANG UNIVERSITY



Part 1

软件

- 软件的概念及特点
- 软件的分类

Part 1 主要内容

- 软件的基本概念
- 软件的特点
- 软件的质量
- 软件的分类

软件的地位和作用

- **深入到社会、经济、生活的方方面面，无处不在**
- **作为一种创新的工具，深刻地改变各个行业和领域，创新问题解决模式和方式，影响人们日常学习、生活和工作**
  - ✓“12306”改变了购票方式，淘宝和京东改变了购物方式
- **软件是实现行业和领域进行信息化融合和改造，实现创新性发展的使能技术和重要利器**
  - ✓企业**信息化改造**，图书借阅等
- **软件已经成为人类社会的关键性基础设施**
  - ✓就像电力、交通设施一样，发挥基础性的作用

建设狗窝和建设大厦的区别

❑搭建狗窝的特点

- ✓简单
- ✓质量要求不高
- ✓一个人就可完成



❑建设大厦的特点

- ✓复杂
- ✓质量高要求很高
- ✓需要团队合作才能完成



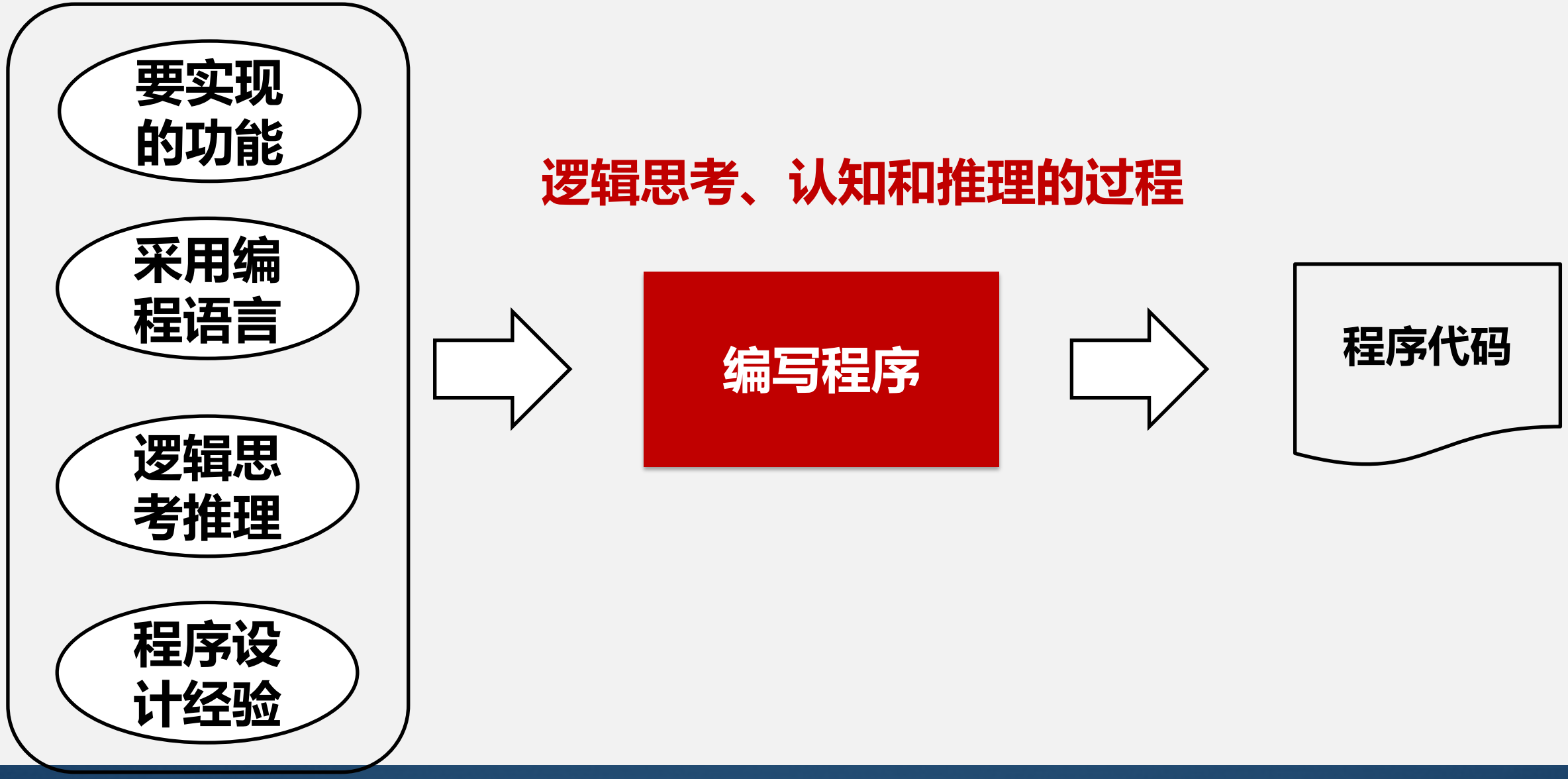
算法 vs 程序 vs 软件

- **算法（Algorithm）解决问题的逻辑步骤和方法，与具体语言无关。它是抽象的思维产物，可以用自然语言、流程图、伪代码来描述。**
  - ✓例：冒泡排序的比较交换逻辑
- **程序（Program） 用某种编程语言实现算法的代码。算法落地成可执行的文本，有了具体语言和语法。**
  - ✓例：用 Python 写的冒泡排序代码
- **软件（Software）由一个或多个程序，加上配置文件、文档、资源等组成的完整可交付系统，面向用户解决实际问题。**
  - ✓例：一个排序工具 App

何为程序?

- **程序（Program）是由程序设计语言所描述的、能为计算机所理解和处理的一组语句序列**
  - ✓用程序设计语言（Programming Language）来描述的
  - ✓如Java、C、C++、Python
- **程序严格遵循程序设计语言的各项语法和语义规定**
  - ✓确保程序代码能为程序设计语言的编译器所理解，进而编译生成相应的可运行代码
- **程序代码可表现为二种形式**
  - ✓**源代码**（Source Code）：用程序设计语言所描述的代码
  - ✓**可执行代码**（Executable Code）：可执行的二进制或中间码

程序如何编写?



软件开发难点与挑战

目标领域类型众多

金融、信息技术、医疗

利益相关者众多

领域专家、设计师、架构师、开发工程师、测试工程师、运维工程师

编程语言众多

C C++ Java Python Dart Go Rust

运行平台众多

Web / Desktop (Windows/Linux/Mac) / Mobile(IOS/Android)

系统类型众多

信息系统、嵌入式系统、CPS、区块链、AI使能系统



挑战：软件开发复杂度

- **数量越来越多**
- **规模越来越大**
- **需求越来越广**
- **成本越来越高**

从程序到软件-工程开发的启发

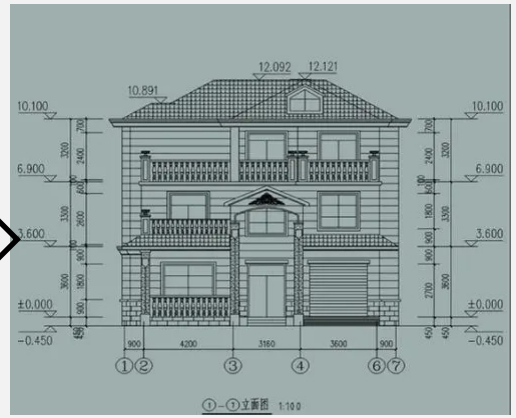
应用
(如火车票购买、酒店预订、网上购物)

编写程序

通过一个步骤就将软件写出来不现实



了解需求

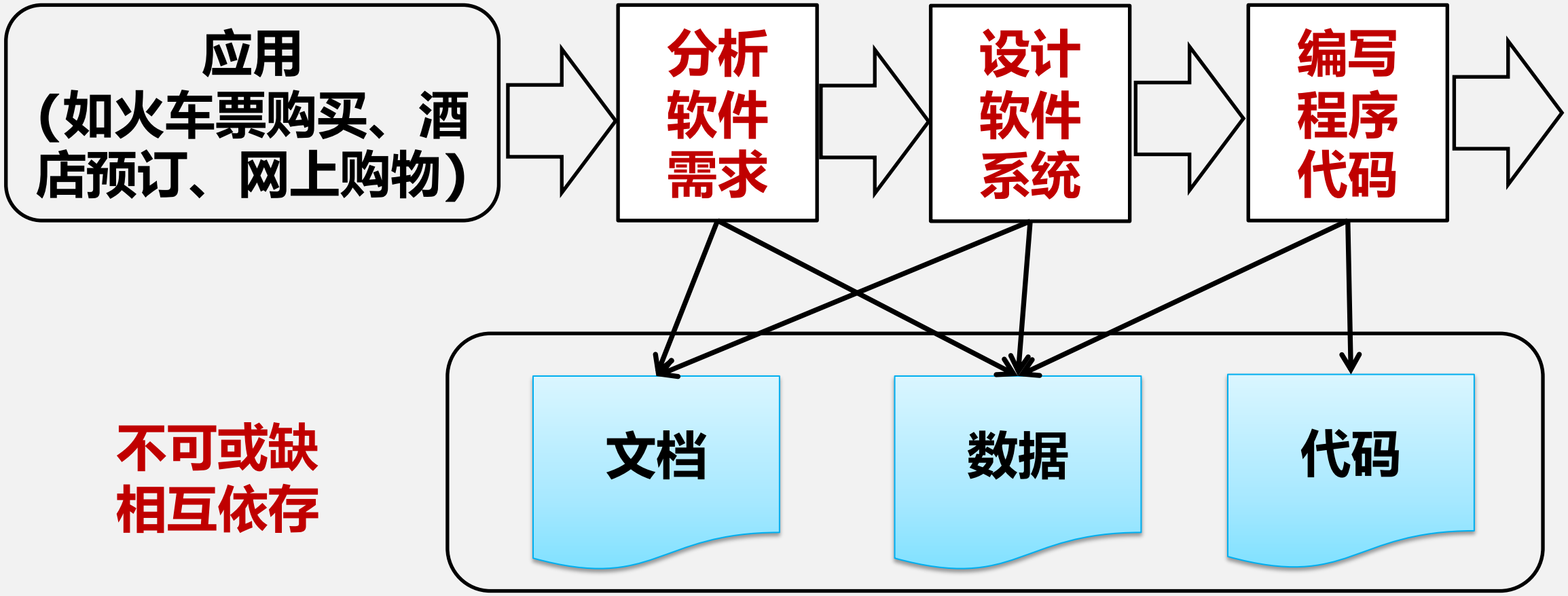


设计图纸



进场施工

编写出程序需要经历诸多的步骤



要编写出高质量的程序需要循序渐进地开展工作

软件开发的再认识

- 开发具有**一定规模**和**复杂性**的**软件系统**与编写一个**简单**的**程序**完全不同不一样
  - ✓ 建设简单平房　vs　建设高楼大厦
  - ✓ 挖一条小沟　vs　建设一条南水北调渠道





软件的**规模**和**复杂性**意味着要采用行之有效的方法，即**工程方法**

软件开发核心问题

1. 什么是对?
2. 什么是好?

产生与预期一致运行结果



4. 代码编写与系统实现 04

3. 系统详细设计 03

2. 系统架构设计 02

1. 需求分析与建模 01



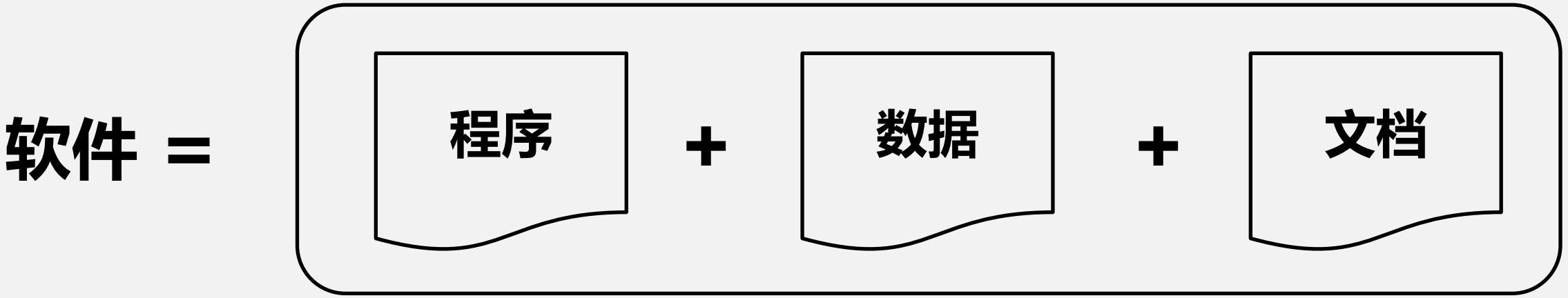
可部署系统代码

软件需求

关键问题：要实现从仅包含系统有限外部信息到完整内部实现信息的分阶段跨越（懂软件工程）

何为软件?

- **软件**是指在计算机系统的支持下，能够完成特定功能与性能的**程序**、**数据**和相关**文档**

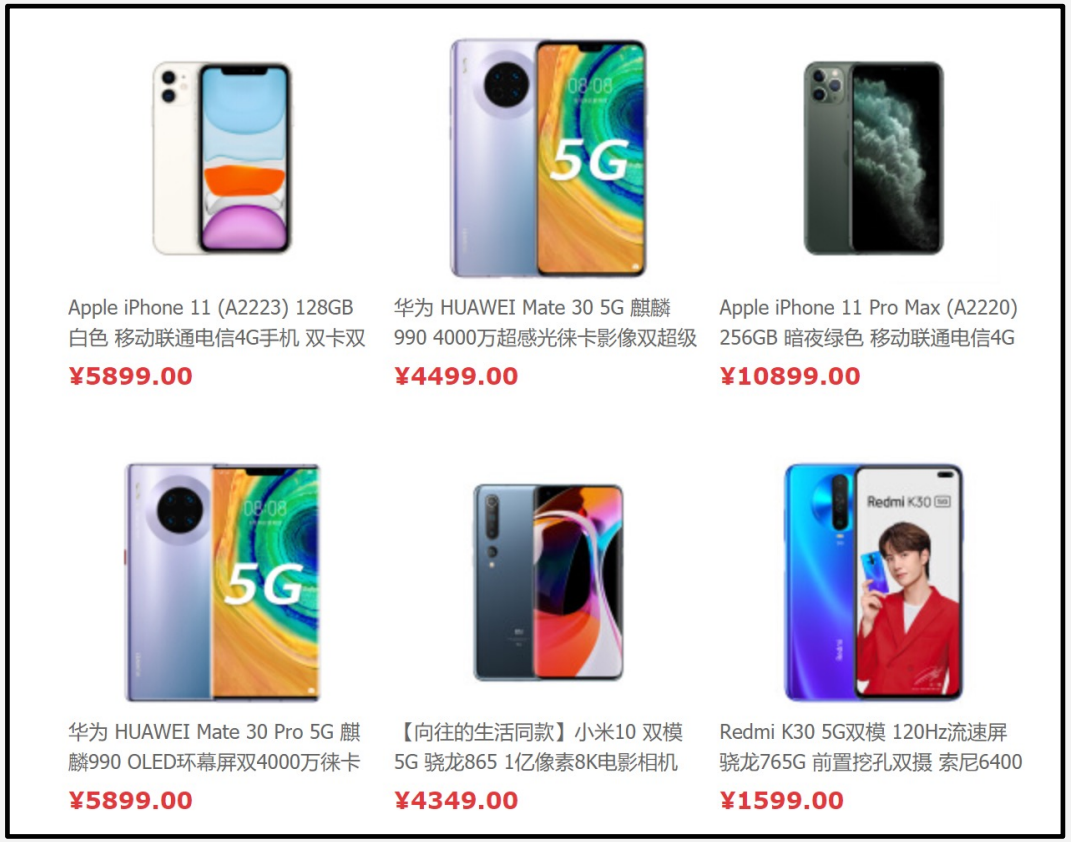


- ✓数据 == 初始化数据 + 测试数据 + 日常数据
- ✓文档 == 开发文档 + 管理文档 + 产品文档

何为数据?

❑数据是程序加工处理的对象和结果

- ✓需要处理哪些数据
- ✓**如何获得**这些数据
- ✓**如何表示**这些数据
- ✓**如何存储**和**检索**数据
- ✓**如何传输**数据



网上商城中的手机及其数据

数据示例

❑数据形式

✓用户、订单、交易、日志数据

❑数据处理

✓表示、获取、存储、检索、分析



何为文档?

- **记录软件开发活动和阶段性成果、软件配置及变更的阐述性资料**
  - ✓定义和理解软件
  - ✓记录软件开发成果
  - ✓辅助不同人员间的交流

- ✓ **软件需求文档**
- ✓ **软件设计文档**
- ✓ **软件测试文档**
- ✓ **软件用户手册**
- ✓ **……**

“空巢老人看护系统”软件系统的需求构思及描述

阐述性资料

1. 背景介绍

随着人口老龄化问题越来越严重，空巢老人的无人看护将成为一个亟待解决的社会难题。家属外出工作时，经常会惦记独自在家的老人，希望掌握老人的状况（比如：身体状态如何？在做什么？）。但是由于空间距离上的限制，家属不能方便快捷的获取这些信息；很多老年人不会或者不方便使用手机，造成家属与老人之间语音通话上的困难；老年人记忆力普遍衰退，容易忘记服药等一些重要的事情，家属外出时无人监督老人完成这些事情。对于一些患有疾病的老人来说，忘记服药很可能会造成危险；当老人独自在家时，他们可能会遇到摔倒或者中风等突发情况，而外出工作的家属和医疗救护人员不能及时掌握这些危险信息，导致救援不及时。

本软件专为解决空巢老人的看护问题而设计，它可以帮助用户（老人的家属）监控老人在家的状况，帮助老人与家人语音聊天，提醒并监督老人按时服药，并在发生危险时向用户警报或者联系急救部门。

为什么需要文档?

❑原因

- ✓软件开发牵涉编程之外的**多方面工作**
- ✓任何事都记在脑子里，**记不住**
- ✓任何事都放在脑子里，**理不清**
- ✓任何事都存在脑子里，**讲不明**

❑编写文档目的

- ✓**阐述清楚**：内容、逻辑性、条理性
- ✓**发现问题**：完整、一致、矛盾
- ✓**开展交流**：便于各种人员的交流
- ✓**促进管理**：管理软件开发成果

Agent工具生成能力对比

案例	介绍	总用例数
ATM	为银行客户提供取款和入账功能	6
TakeOut MS	为消费者提供了一个便捷的方式，通过在线系统从他们喜爱的多种美食中选择，并对用餐体验进行评分，帮助企业提升服务质量。	8
LoanPS	涵盖提交和评估贷款申请、预订新贷款以及付款等应用场景。	10

Agent工具生成能力对比

		集成测试	单元测试	错误迭代次数
ATM	openspec	62.5%	85.2%	6
	speckit	67.5%	81.25%	7
	claude	50%	75%	3
TakeOutMS	openspec	100%	93.75%	2
	speckit	50%	80%	4
	claude	75%	85%	2
LoanPS	openspec	72.4%	85%	3
	speckit	79.3%	87.9%	4
	claude	68.9%	81.3%	2

集成测试：验证多个模块之间的接口与交互是否协同工作
功能测试：验证软件系统的单个功能或操作是否符合预期，针对单个具体接口或业务规则进行验证

	平均集成测试成功率	平均单元测试成功率	平均错误迭代次数
原生 claude	64.63%	80.43%	2.33
spec-driven工具(openspec,speckit)	72.03%	85.52%	4.33

对于三种不同规模的例子，openspec和speckit两种规范驱动工具在微服务项目生成上的集成测试、单元测试成功率均高于原生claude，且随项目编码复杂程度的升高，优势越明显。

文档的重要性

- **文档在软件工程中特别重要**，文档是否规范与齐全，**是衡量软件企业是否成熟的重要标志之一**
- **软件文档分类**
  - ✓ **开发文档**：主要由项目组书写，用于指导软件开发与维护
  - ✓ **管理文档**：主要由软件工程管理部门书写，用于指导软件管理和决策
  - ✓ **产品文档**：参考手册和用户指南等
- 软件工程规定：**文档必须指挥程序**，而决不允许程序指挥文档；**文档与程序**必须保持**高度一致**，而决不允许程序脱离开文档。

开发文档

开发文档具有严格的层次关系：

(1) 《目标程序》覆盖 《源程序》

(2) **《源程序》**覆盖 **《详细设计说明书》**

(3) **《详细设计说明书》**覆盖 **《概要设计说明书》**

(4) **《概要设计说明书》**覆盖 **《需求规格说明书》**

(5) **《需求规格说明书 》**覆盖 **《用户需求报告》**

(6) 《用户需求报告》覆盖 《软件合同》

软件文档示例

空巢老人智能看护系统设计文档

1、引言.....4

1.1 编写目的.....4
1.2 读者对象.....4
1.3 软件项目概述.....4
1.4 文档概述.....5
1.5 定义.....5
1.6 参考资料.....5

2、软件设计约束.....5

2.1 软件设计目标和原则.....5
2.2 软件设计的约束和限制.....6

3、软件设计.....6

3.1 软件体系结构设计.....6
3.2 用户界面设计.....9
3.3 用例设计.....13
3.4 类设计.....24
3.5 数据设计.....32
3.6 部署设计.....33

软件文档规范



Captured with Xnip

ICS 35.080
L 77

中华人民共和国国家标准

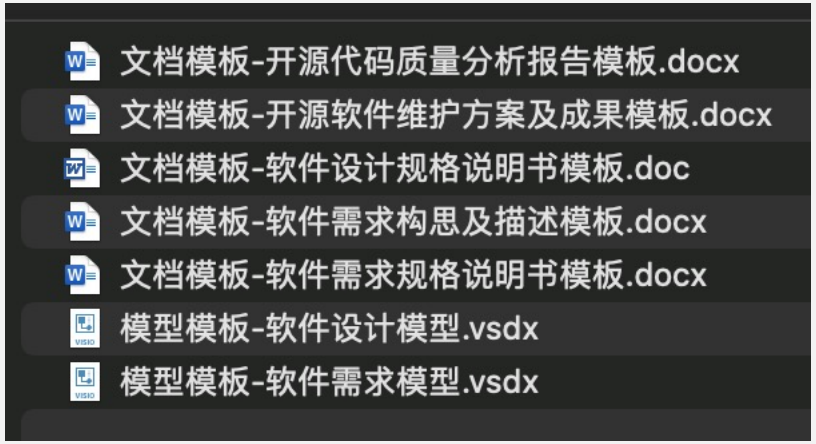
GB/T 8567—2006
代替 GB/T 8567—1988

计算机软件文档编制规范

Specification for computer software documentation

2006-03-14 发布　　2006-07-01 实施

中华人民共和国国家质量监督检验检疫总局
中国国家标准化管理委员会　发布



参考模版



能不能改进现有Spec-driven工具，产生符合标准的文档?

Part 1 主要内容

- 软件的基本概念
- 软件的特点
- 软件的质量
- 软件的分类

主观题 10分

设置

软件特点有哪些?



Q1：与硬件系统（如手机、电视机）相比较，软件系统有何特殊性?

Q2：它们给软件系统的开发和维护带来什么样的新挑战?



作答

软件的特点小结

❑逻辑性

- ✓**逻辑产品**，思维活动（而非物理活动）的结果、不会磨损和老化

❑设计开发

- ✓是**设计开发**而成的，不是生产制造而成的

❑易变性

- ✓需求**经常变**、难以把控，影响软件的制品即开发过程

❑复杂性

- ✓**规模大**：代码行、模块、接入人员、进程、数据等数量非常大
- ✓**运行复杂**：状态很难追踪和复现

❑缺陷的隐蔽性

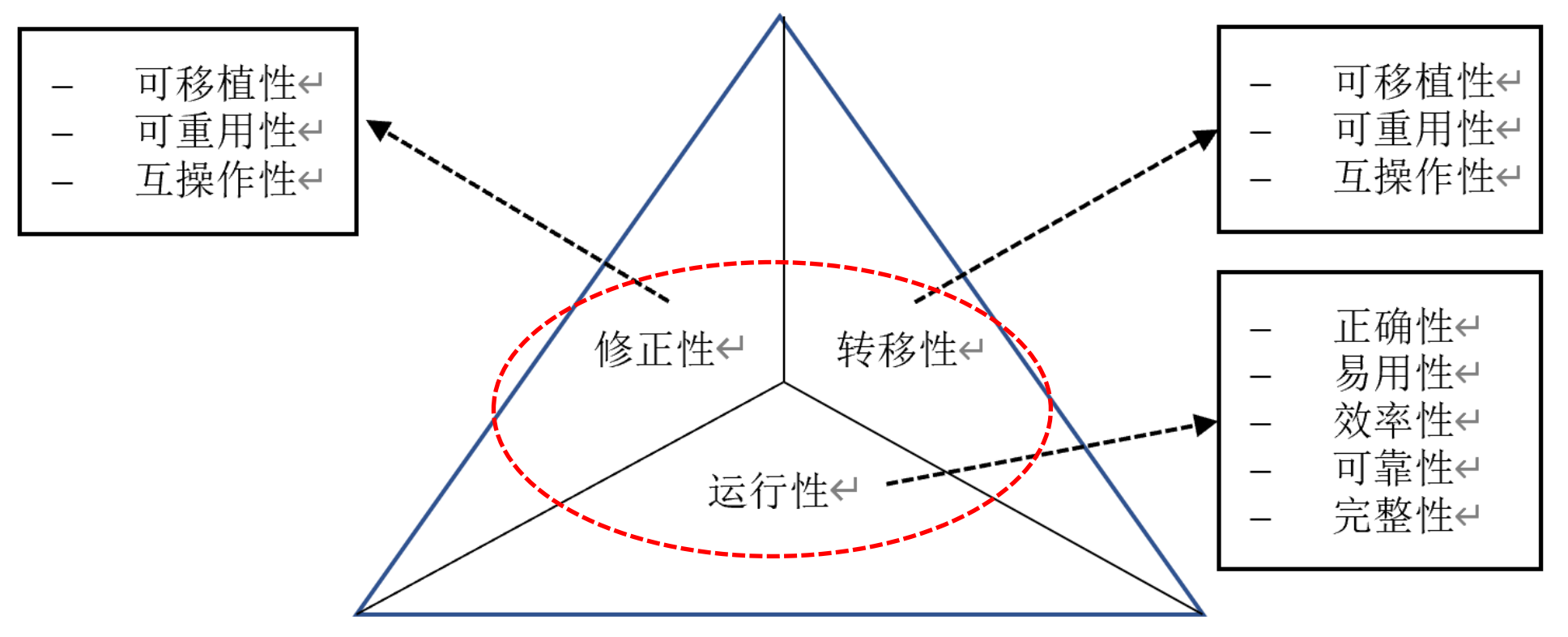
- ✓缺陷隐藏在逻辑代码中，不像硬件系统那样直观显现，很难被人们所发现和排除

Part 1 主要内容

- 软件的基本概念
- 软件的特点
- 软件的质量
- 软件的分类

软件质量

❑软件质量是指软件满足给定需求的程度，它是产品生命线



软件质量要素（1/3）

❑正确性（Correctness）

✓软件**满足规格说明和用户要求**的程度，即在预定环境下能正确地完成预期**功能**的程度。

❑可靠性（Reliability）

✓在规定条件下、限定时间范围内，软件系统完成预期功能、**不引起系统故障**的能力。

❑健壮性（Robustness）

✓在计算环境发生故障、**输入无效数据或操作错误**等意外情况下，软件仍能做出适当响应的程度。

❑有效性（Efficiency）

✓软件利用计算资源和存储资源以实现其功能的能力。

软件质量要素（2/3）

- **安全性**
  - ✓包括**系统安全（Safety）**和**信息安全（Security）**，系统安全是指软件能及时有效地避免给人员、设施、环境、经济等造成损害，信息安全是指软件能有效防控各类的非法获取、传播和使用
- **可维护性（Maintainability）**
  - ✓是否**易于**对其进行修改以**更正错误、增强功能**、适应新运行环境
- **可移植性（Portability）**
  - ✓把软件从**一种运行环境**转移到**另一种运行环境**下运行的**难易程度**
- **可重用性（Reusability）**
  - ✓软件模块、构件、设计方案等在其他软件开发中**被再次使用程度**

软件质量要素（3/3）

- **可理解性（Comprehensibility）**
  - ✓软件开发者或用户**理解**该软件系统的**容易程度**
- **可信性（Dependability）**
  - ✓采取有效措施确认软件满足人们的要求和期望
- **持续性（Sustainability）**
  - ✓面对各种突发异常事件，仍能提供令人满意的服务的能力
- **易用性（Usability）**
  - ✓使用和操作软件系统**难易程度**
- **互操作性（Interoperability）**
  - ✓软件系统与其他的系统进行交换信息、协同工作的能力

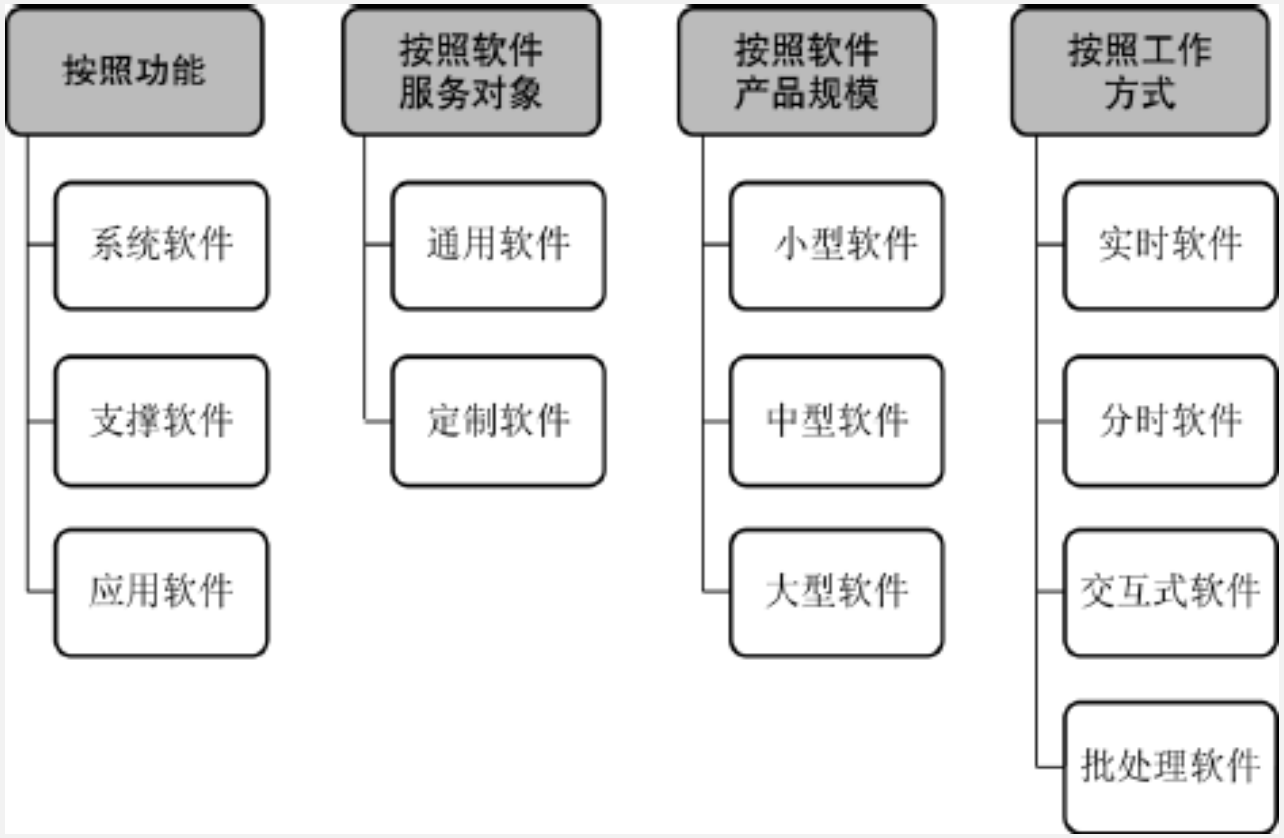


能不能改进现有Spec-driven工具，产生高质量代码?

Part 1 主要内容

- 软件的基本概念
- 软件的特点
- 软件的质量
- 软件的分类

软件分类



- **关键基础软件**
  - 数据库
  - 操作系统
  - 编译器
- **大型工业软件**
- **行业应用软件**
- **新型平台软件**
  - 云计算系统
  - 智能计算系统
- **嵌入式软件**

按照功能分类

❑系统软件

- ✓ 对计算机资源进行管理，**为应用软件的运行提供基础设施和服务**的一类软件。从计算服务的视角，系统软件介于计算机硬件和应用软件之间。
- ✓ 示例：操作系统、数据库管理系统、编译软件、软件中间件等

❑支撑软件

- ✓ **辅助软件开发和运维**，帮助开发人员完成软件开发和维护工作的一类软件
- ✓ 示例：SonarQube、Visual Studio、Eclipse等

❑应用软件

- ✓ 面向**特定应用领域**的专用软件。它们针对相关行业和领域的特定问题，为其提供基于计算的新颖解决方案。
- ✓ 示例：淘宝、12306、携程、微信、QQ等软件

开源软件

❑何为开源软件

✓一种源代码可以**自由获取和传播**的计算机软件，其拥有者通过开源许可证赋予被许可人对软件**进行使用、修改和传播**

❑开源软件的好处

✓源代码可自由传播

✓激发创作者的热情

✓免费使用降低成本



开源软件许可证

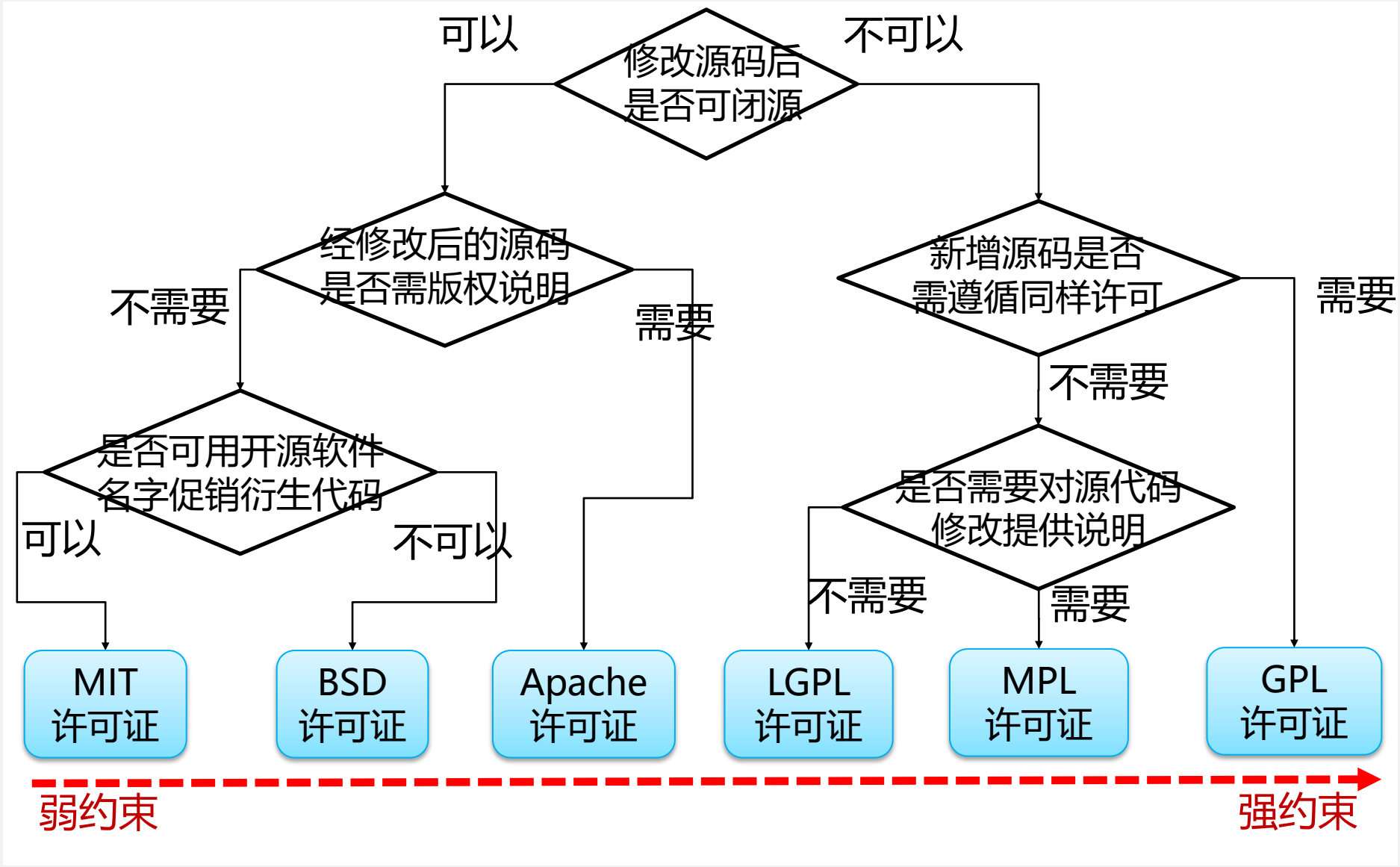
❑何为开源许可证

- ✓**声明获得开源代码后拥有的权利，界定对别人的开源作品进行何种操作、何种操作是被禁止的，规范开源软件的使用要求和约束**
- ✓开发者虽然可自由地获取开源软件的源代码，但在如何使用开源软件方面还需要遵循相关的开源软件协议

❑为什么需要开源许可证

- ✓尝试在开源软件的**自由创新**与**创业利益**之间达成某种**平衡**
- ✓既支持开发者基于开源软件进行创新，也**保护贡献者和创新者的相关利益**，同时寻求某些商业运作模式，促进开源软件长期、持续和良性的发展

不同许可证的差别


修改源码后
是否可闭源
可以
不可以
经修改后的源码
是否需版权说明
不需要
需要
新增源码是否
需遵循同样许可
需要
不需要
是否可用开源软件
名字促销衍生代码
可以
不可以
是否需要对源代码
修改提供说明
不需要
需要
MIT
许可证
BSD
许可证
Apache
许可证
LGPL
许可证
MPL
许可证
GPL
许可证
弱约束
强约束

主观题 10分

设置



- **开源给智能软件开发带来什么样的好处?**

作答

北京航空航天大學
BEIHANG UNIVERSITY



Part 2

软件演化及危机

- 软件危机的表现与原因
- 软件危机的启示

Part 2 主要内容

- 软件发展与挑战
- 软件危机与灾难
- 软件危机的根源与启示

Part 2 主要内容

- 软件发展与挑战
- 软件危机与灾难
- 软件危机的根源与启示

纵观软件发展

- 数量越来越多
- 规模越来越大
- 需求越来越广
- 成本越来越高

❑应用领域变化

✓最早满足军方应用，如科学计算

✓逐步走向商业应用等新领域，如**银行、航空**等领域的事务处理

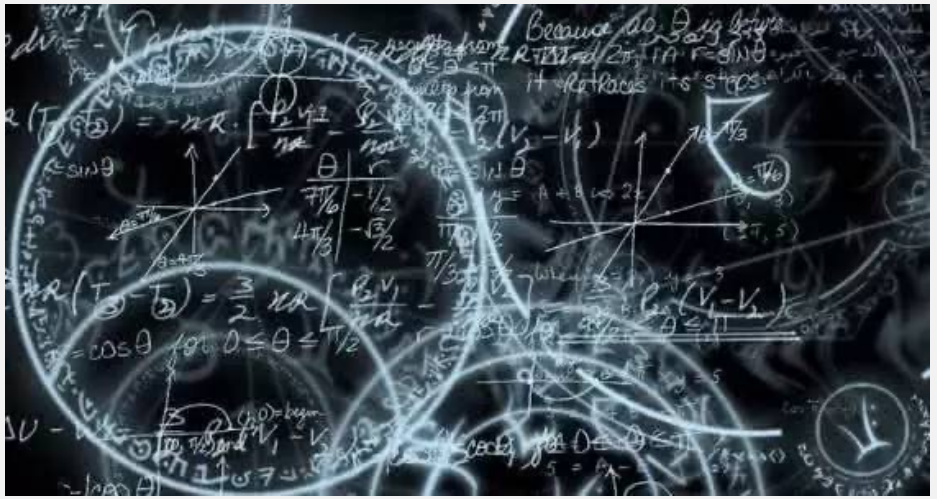
❑应用**数量增长**

✓计算机软件的需求量不断上升

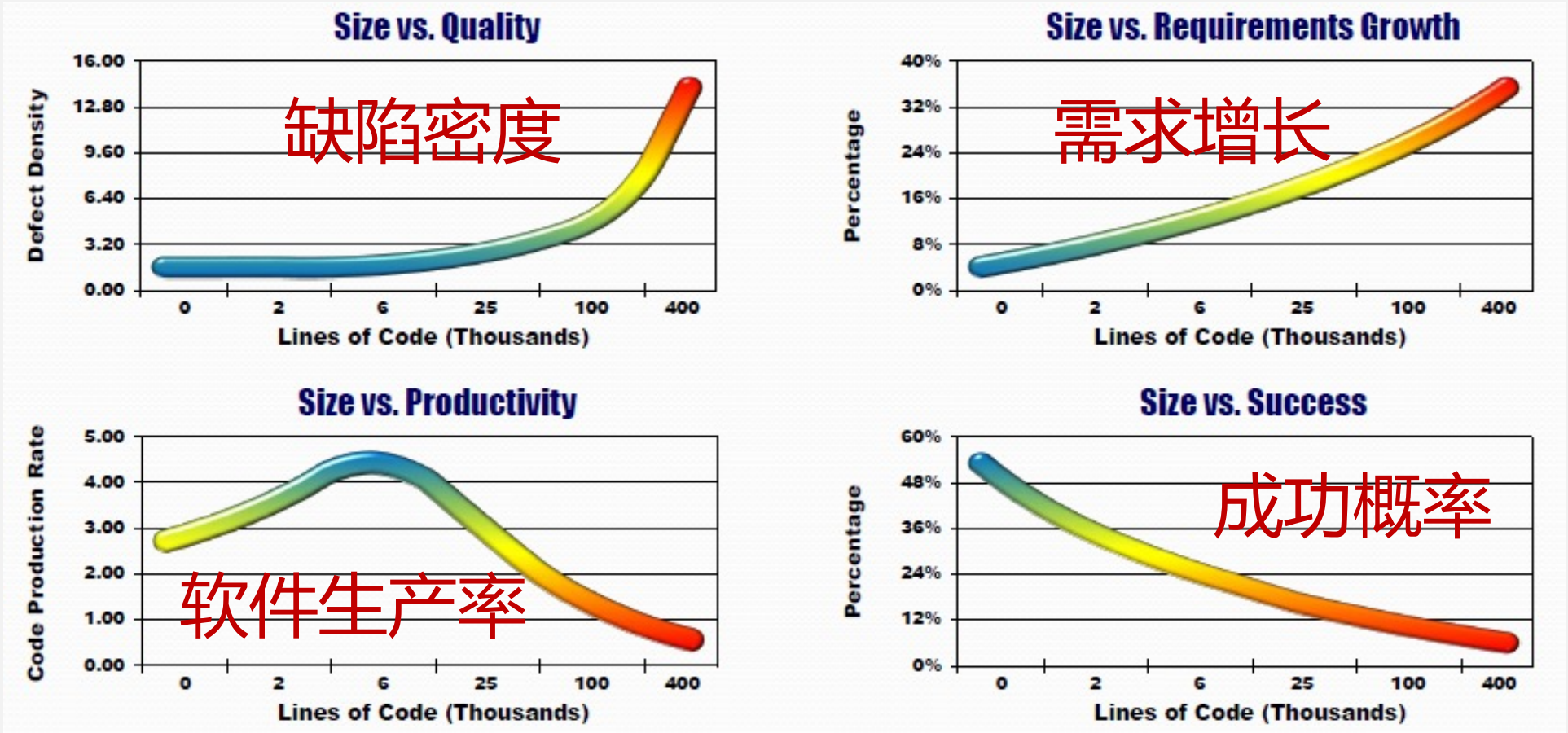
❑应用**复杂性增加**

✓多样化的用户

✓多样化的需求



软件开发面临的挑战日趋突出

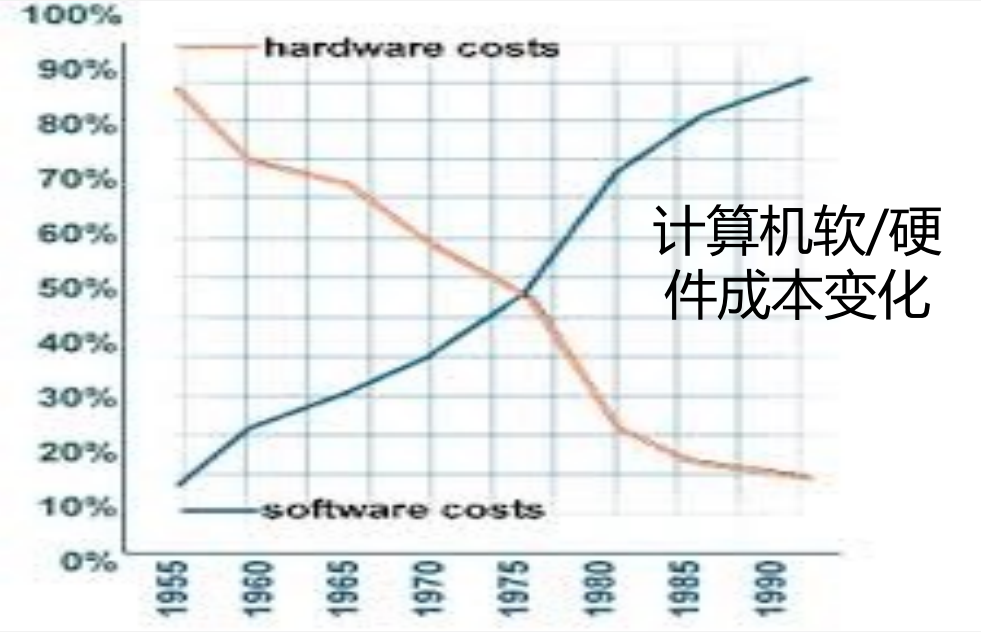


代码规模增长对质量、生产率、成功开发带来的影响

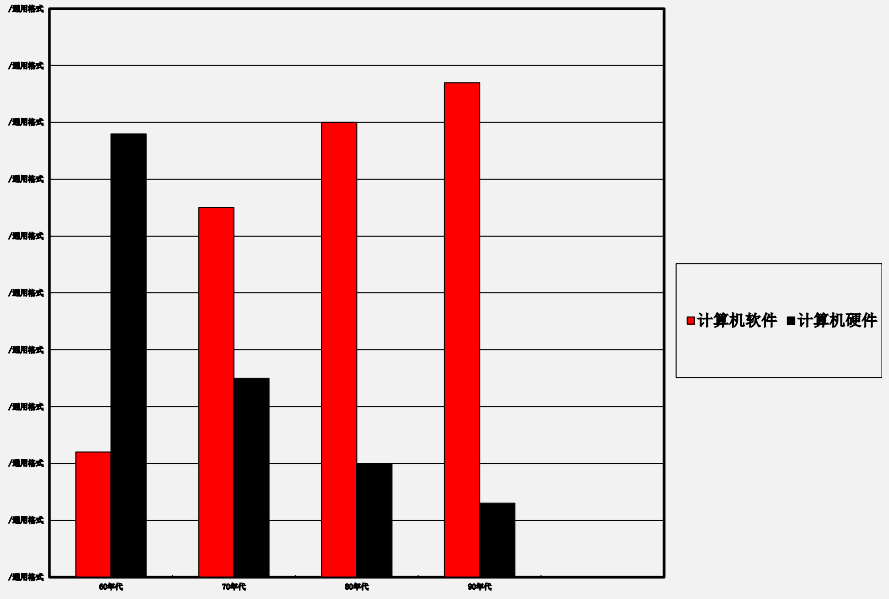
开发成本高

❑软件成本高，软硬件投资比发生急剧变化

- ✓美国空军：1955年软件占总费用(计算机系统)的18%，70年60%，85年达到85%
- ✓IBM 360 OS：5000+人年，耗时4年(1963-1966)，花费2亿多美元



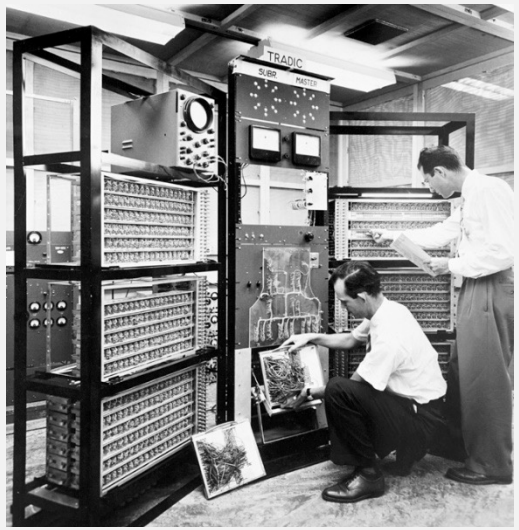
计算机软/硬件成本变化



Part 2 主要内容

- 软件发展与挑战
- 软件危机与灾难
- 软件危机的根源与启示

传统的个体作坊式软件开发







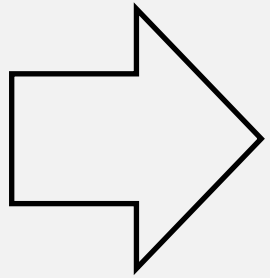
作坊式的个人创作

第二代晶体管计算机：
TRADIC（1954）
IBM 1401（1958）

- 依靠个人的能力
- 相互之间缺乏合作
- 关注计算存储时空利用，精雕细琢
- 程序规模小且功能单一
- 无系统性方法和支撑工具

个体作坊式创作带来的问题

作坊式的个体编程开发



大批量和大规模软件系统的开发

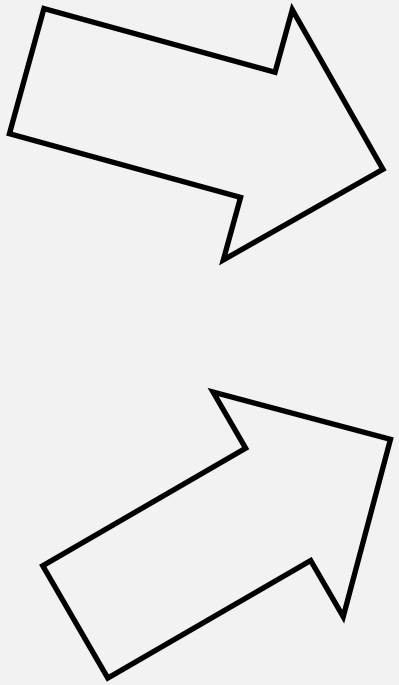


个体作坊式方法会给软件开发带来什么问题?

软件危机的出现

作坊式的个体编程

大批量、大规模软件开发



软件危机

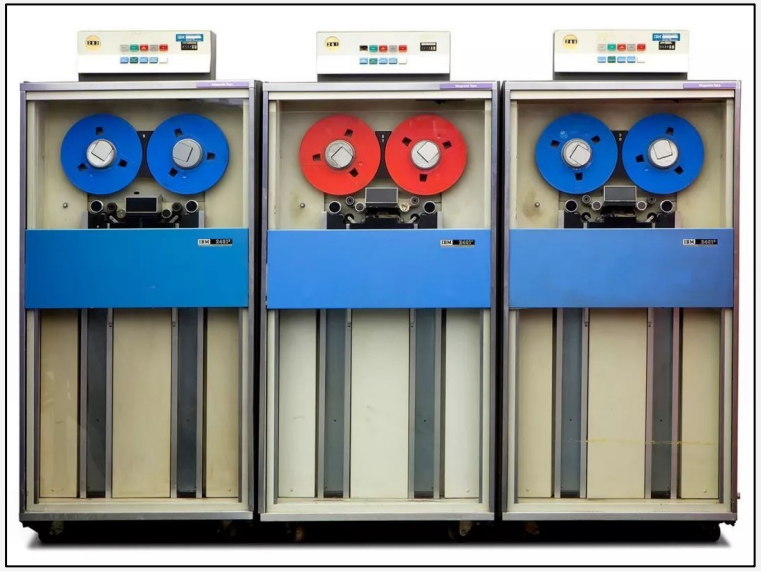
- ✓ **进度**经常延迟
- ✓ **质量**无法保证
- ✓ **成本**超出预算
- ✓ 软件**维护**困难
- ✓ 失败**风险**很大

软件危机

- **软件工程来源于软件危机 - 先有软件危机，后有软件工程**
- 二十世纪六十年代中期，在美国出现了软件危机（Software Crisis）
  - ✓定义：软件开发和维护过程中所遇到一系列难以控制的问题
  - ✓1968年北大西洋公约组织（NATO）的计算机科学家在联邦德国召开的国际学术会议上第一次提出了“软件危机” 这个名词
- 软件危机表现在**研发大型软件**时，软件开发的**成本增大、进度延期、维护困难和质量得不到保障**
- **如何开发软件，怎样满足对软件日益增长的需求**

示例：IBM 360 OS软件开发







- OS/360 超大型软件项目(1960s初)：
  - 复杂软件：支持多道程序，最多可同时运行15道程序
- 软件工程师**超2000人**，花费**超5亿美元**，工作量**超5000人/年**
  - 有史以来最可怕的**软件开发泥潭**
- Brooks，《人月神话》The Mythical Man-Month、图灵奖获得者



软件维护困难

- **理解**
  - ✓读懂程序比较困难，尤其是他人程序
- **修改**
  - ✓程序非常脆弱，牵一发而动全身
- **出错**
  - ✓改了以后易引入错误
- **发现**
  - ✓有了错误后难以发现

```
1  import jade.core.Agent;
10
11 public class MiningAgent extends Agent
12 {
13     private final static int EMPTY = 0;
14     private final static int GOLD = 1;
15     private final static int OBSTACLE = 2;
16     private final static int HOUSE = 3;
17     private final int DETECT = 4;
18     private final int EXPLORE = 5;
19     private final int PROVIDE = 6;
20     private final int SIZE = 15;
21
22     private MarUI ui = null;
23     private Gold gold = null;
24
25     public OneRoleAgent()
26     {
27         ui = MarUI.getUI();
28         gold = Gold.getGold();
29     }
30
31
32     public void setup()
33     {
34         ui.runInfo.setText(ui.runInfo.getText()+"加载采矿机器人. . . . "+"\n");
35         addBehaviour(new DetectGold());
36     }
37
38     public static Coordinate makeTarget(int[][] myMap ) {
39         int i = 0, tx =0, ty = 0;
40
41         for (; i < 1;) {
42             tx = (int) (Math.random() * 15);
```

质量难以保证

- 人总是会犯错误的
- 软件开发的错误表现为多种形式
  - ✓没有按照要求（需求）来开发
  - ✓编写的代码在功能上存在错误
  - ✓实现了功能但是性能达不到要求
  - ✓所开发的软件交互界面用户不喜欢
  - ✓……
- 有些软件错误可能是致命的

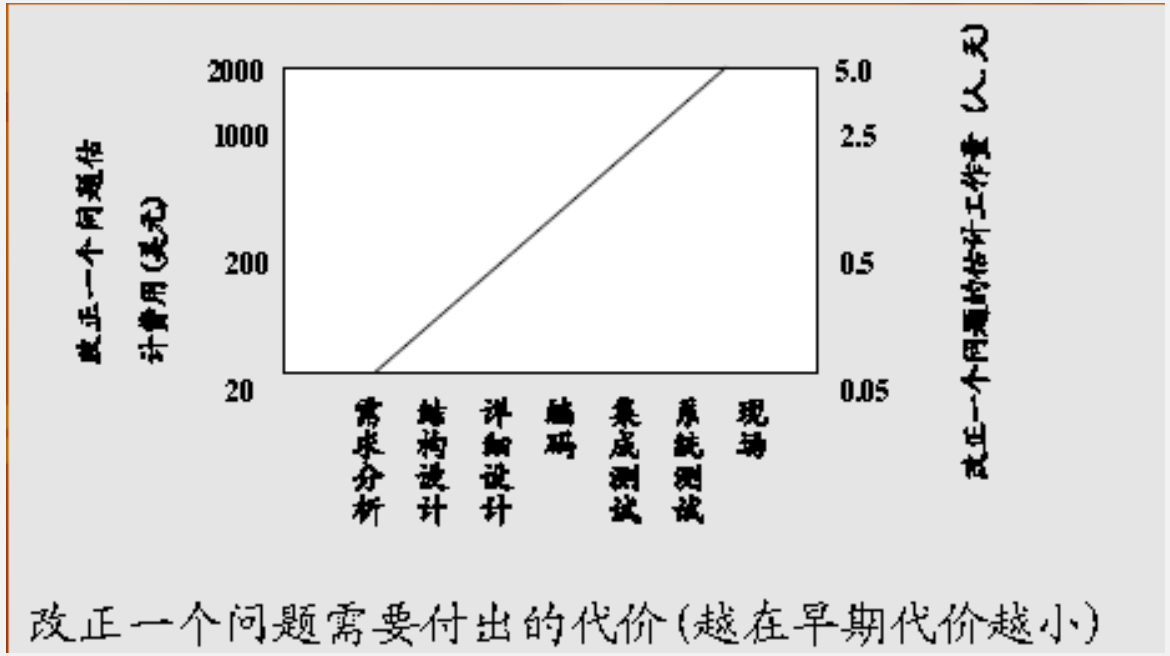




原软件用于教练飞机某参数屏显范围±100，重用于新战斗机，该参数屏显范围应该为±300！

软件错误后果

- **软件错误的普遍存在，修正代价高**
  - ✓ 软件人员编制 1000 行程序，平均产生错误 20-50 个
  - ✓ 美国航天飞机开发软件的代价，软件平均一行程序花费 1000 美元
- **考虑潜在软件失效原因越多、改正越早，系统就越不易出现失效**



改正一个问题需要付出的代价(越在早期代价越小)

软件灾难故事

ARIANE 5 火箭

–1996年6月，耗资70亿美元，发射37秒后爆炸

– 发射失败的原因在于软件的错误

– 程序中试图将64位浮点数转换成 16位整数时产生溢出

– 缺少错误处理程序对数据溢出进行管理

– 严格地遵守软件验证过程可以避免这种错误



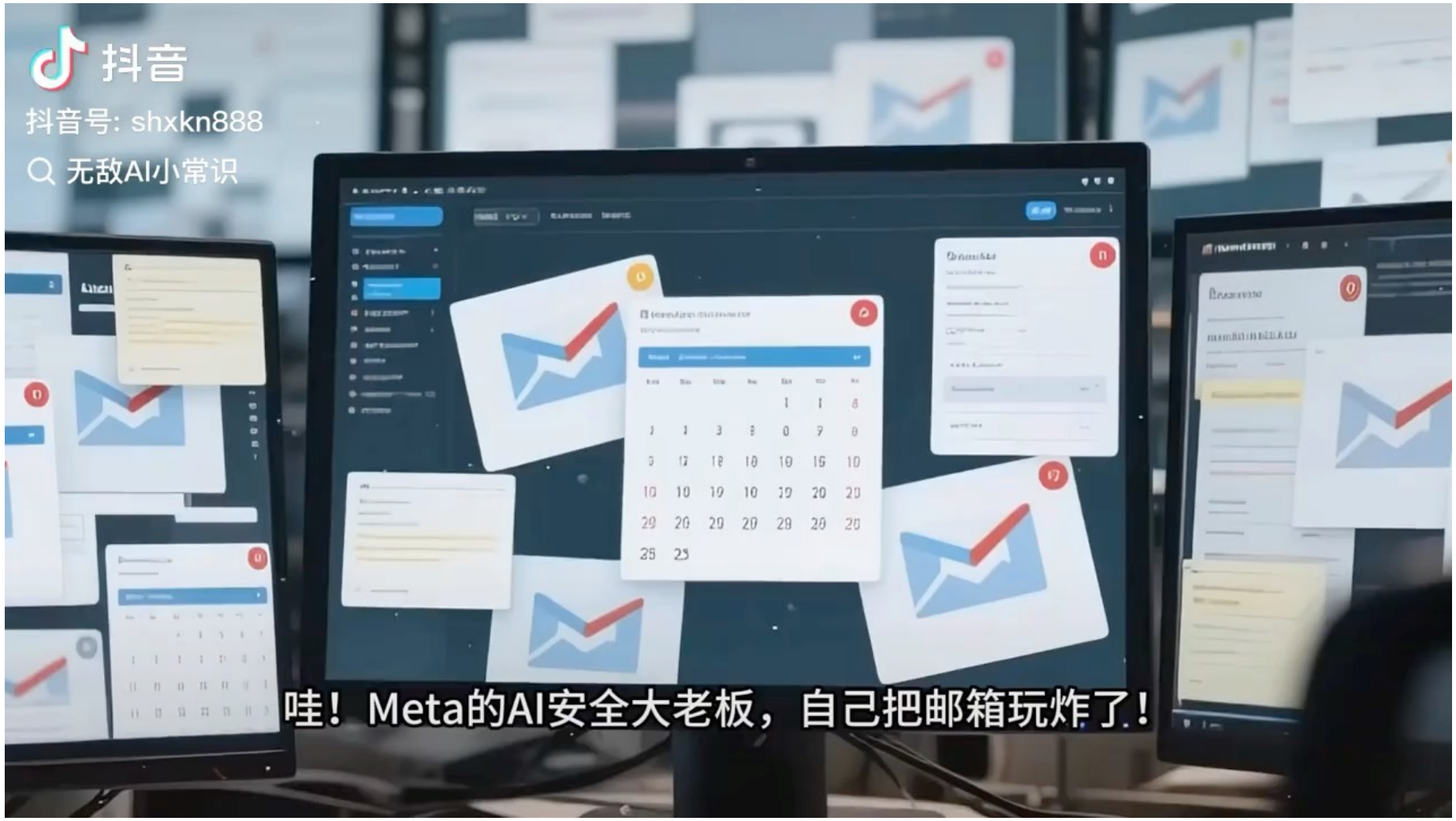
软件灾难故事 – 特斯拉


视频内容均为行车记录仪视频
抖音
抖音号: GeekLaii
前
后
左
右
特斯拉
主动安全
Autopilot
失败案例集锦

软件灾难故事 – 737 Max



大模型灾难故事


抖音
抖音号: shxkn888
无敌AI小常识
哇！Meta的AI安全大老板，自己把邮箱玩炸了！


抖音
抖音号: 1567949856
Lif
Tom's Hardware
崩了
13h
AWS的AI编码工具
把自家云服务搞崩了
AI事故
云服务
AI自行决定删除整个环境重建

Part 2 主要内容

- 软件发展与挑战
- 软件危机与灾难
- 软件危机的根源与启示

软件危机产生根源总结

- **对软件这样一类复杂和特殊系统的认识不清**
  - ✓软件是新生事物，对其特点、规律性和复杂性认识不够
- **没有找到支持软件系统开发的有效方法**
  - ✓基础理论、关键技术、开发过程、支撑工具等
- **缺乏成功软件开发实践以及相应的开发经验**
  - ✓系统总结、认真分析、充分借鉴、吸取教训

软件开发迫切需要理论和方法指导，软件工程应运而生！

解决危机的途径

❑ 技术措施

使用更好的软件开发技术、开发方法和开发工具

❑ 组织管理措施

（1）创造良好的组织、严密的管理与协调工作的机制。软件开发不是某种个体劳动的神秘技巧，而应该是一种组织良好、管理严密、各类人员协同配合、共同完成的工程项目。

（2）摆脱软件危机的主要出路是，**按工程化的原则**和方法组织软件的开发工作。

（3）**强调文档的重要性。**“口说无凭，立字为据！！”是解决软件危机的格言。

学好软件工程，获得应对危机的基本能力

Claude Sonnet 4.6 的回答

先回顾软件危机的本质（1960s-70s）

软件危机的根源不只是"写代码慢"，而是：

1. **复杂性管理失控** — 系统规模超出人脑理解能力
2. **需求不明确** — 用户说不清要什么
3. **缺乏工程方法** — 没有成熟的开发流程、架构模式
4. **质量无法保证** — 测试、验证体系原始
5. **进度/成本失控** — 估算能力极差

大模型能解决什么

危机根源	大模型的帮助
编码效率低	✅ 大幅提升，这是最直接的收益
文档缺失	✅ 自动生成注释和文档
代码复用难	✅ 快速检索和生成模板
入门门槛高	✅ 降低对程序员数量的需求

如果那个年代有 Claude/Copilot，NASA、IBM 的工程师效率可能提升 5-10 倍。

人类在软件上真正的难题，从来都不是打字速度。

Claude Sonnet 4.6 的回答

但解决不了什么

1. 需求本质上是人的问题

> 客户说"我要一个更快的马"，大模型再强也只会造出一匹更快的马。

需求模糊、频繁变更、stakeholder 利益冲突——这是人类组织问题，不是编码问题。

2. 系统复杂性的上限会被推高

大模型让代码生产变得极其容易，反而可能让人们**建造出更复杂、更庞大、更难维护的系统**。就像高速公路缓解了拥堵，却诱导了更多车辆上路（**诱发需求**）。

最可能的历史走向

```
原来的危机：写不够快、人不够多
            ↓ 有了大模型
新的危机：代码泛滥、架构失控、
            无人理解自己系统的全貌
```

本质上，**软件危机催生了软件工程学**（结构化编程、瀑布模型、后来的敏捷）。大模型穿越过去，可能让这个"倒逼进化"的过程被跳过——最终在更大规模上重蹈覆辙。

大模型能把软件危机从“生产力危机”变成“治理危机”— 危机不会消失，只会升级

北京航空航天大學
BEIHANG UNIVERSITY



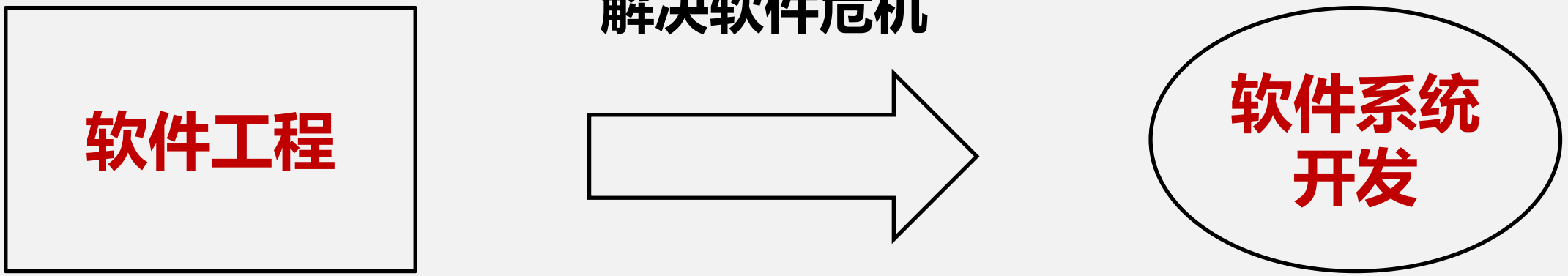
Part 3

软件工程

主要内容

- 软件工程基本概念
- 软件工程发展历史
- 软件工程目标和三要素
- 软件工程的基本原则与规范

软件工程产生的动机

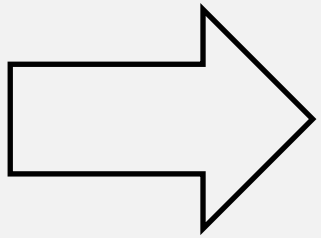


- 快速
- 高效
- 低成本
- 高质量

软件开发方式的改变

- 从个体作坊式行为 ==》基于团队的协同开发方式

作坊式的个体编程创作



基于团队的协同开发

- ✓ 团队协作
- ✓ 分步实施
- ✓ 质量保证
- ✓ 开发技术
- ✓ 支持工具
- ✓ ……

软件工程的定义

- **IEEE 1993 将系统的、规范的、可量化的方法应用于软件的开发、运行和维护的过程；以及上述方法的研究**
  - ✓**系统化**：提供完整和全面的解决方法，包括目标、原则、过程模型、开发活动、开发方法和技术等
  - ✓**规范化**：支持各类软件系统的开发，包括语言标准、质量标准、编程标准、方法标准、能力极其改进标准等
  - ✓**可量化**：工作量、成本、进度、质量等要素可以量化
- **1995年，GB/T11457 T11457-1995《软件工程术语》，“软件工程”被定义为“软件开发、运行、维护和引退的系统方法“**
- **2001年，Roger S. Pressman 对软件工程的定义是：软件工程是一个过程、一组方法和一系列工具。**

软件工程定义

- 在现代，软件工程是指应用计算机科学技术、数学、管理学的原理，运用工程科学的理论、方法和技术，研究和指导软件开发和演化的一门交叉学科，旨在开发满足用户需求、及时交付、不超过预算和无故障的软件
- 软件工程是指开发、使用和维护软件系统的系统、规范、可度量的方法，包括软件需求、软件设计、软件构造、软件测试、软件维护、软件配置管理、软件工程管理、软件工程过程、软件工程工具和方法及软件质量等内容

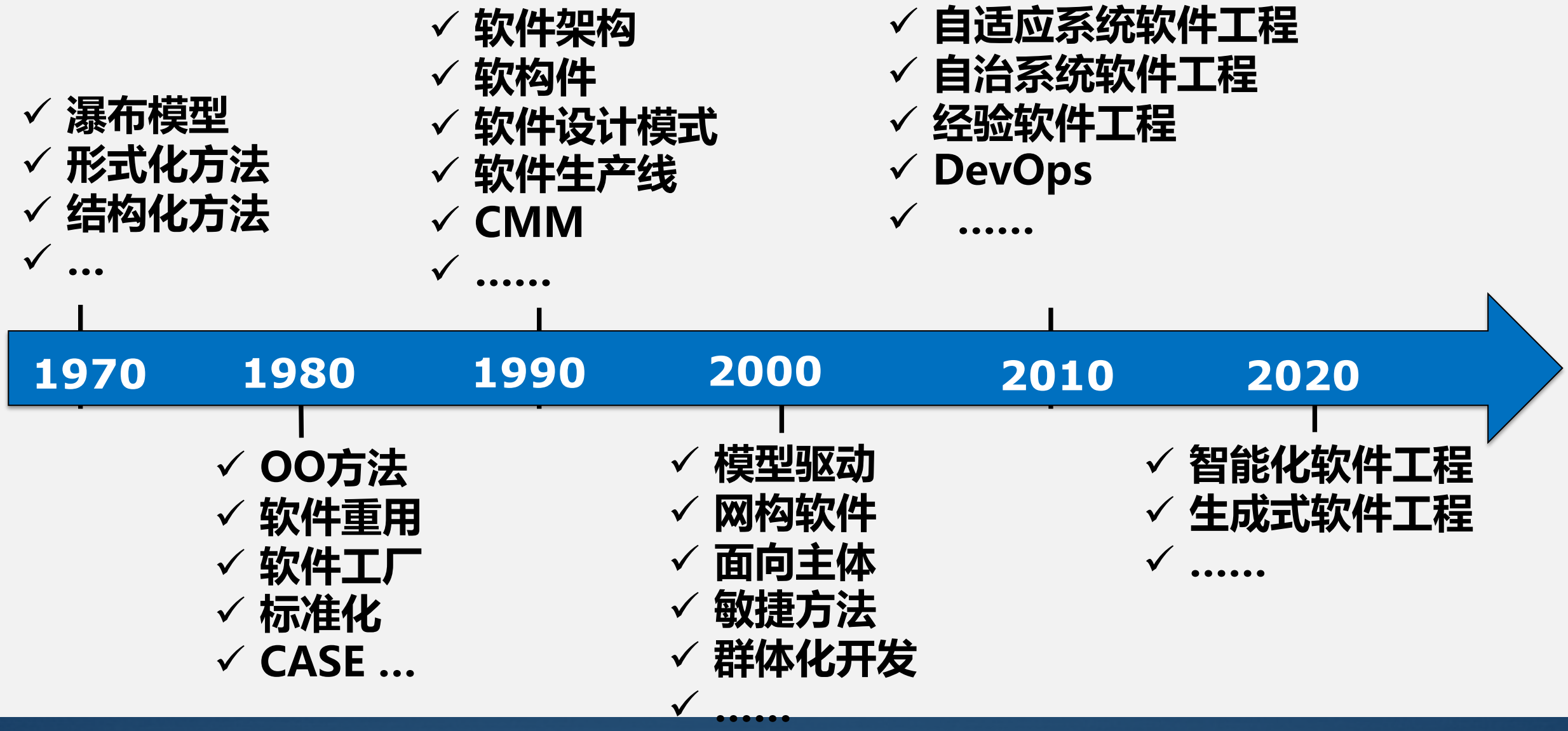
软件工程的作用

- 从软件项目团队来讲，解决在规定的时间内，按照规定的成本，完成预期质量目标（软件的功能、性能和接口达到需求报告标准）的软件。
- 从软件企业本身来讲，解决持续地规范软件开发过程和软件管理过程，不断地优化软件组织的个人素质和集体素质，从而逐渐增强软件企业的市场竞争实力。

主要内容

- 软件工程基本概念
- 软件工程发展历史
- 软件工程目标和三要素
- 软件工程的基本原则与规范

软件工程的发展历程


✓ 瀑布模型
✓ 形式化方法
✓ 结构化方法
✓ …
✓ 软件架构
✓ 软构件
✓ 软件设计模式
✓ 软件生产线
✓ CMM
✓ ……
✓ 自适应系统软件工程
✓ 自治系统软件工程
✓ 经验软件工程
✓ DevOps
✓ ……
1970
1980
1990
2000
2010
2020
✓ OO方法
✓ 软件重用
✓ 软件工厂
✓ 标准化
✓ CASE …
✓ 模型驱动
✓ 网构软件
✓ 面向主体
✓ 敏捷方法
✓ 群体化开发
✓ ……
✓ 智能化软件工程
✓ 生成式软件工程
✓ ……

20世纪50-60年代（1950-1960）

- 软件系统较为简单，计算机软件与硬件结合的非常紧密
- “精雕细琢”程序代码，以充分利用宝贵的计算资源
- 出现了黑客文化，倡导自由
- 成功的案例
  - ✓ IBM OS/360软件系统的成功研制并投入商业使用
- 出现了高级程序设计语言
  - ✓ 如Fortran、COBOL、LISP、ALGOR等
- 这一时期的软件开发手段落后，开发效率低，质量无法保证，进而引发了软件危机

20世纪70年代 （1970 - 1980）

- 计算机主机的**计算能力**得到了很大提升
- 计算机软件朝着商业应用拓展，需要处理繁杂的事务流程
- **程序设计语言和程序设计方法学**成为研究热点
  - ✓ 出现了诸如PASCAL、C、Prolog、ML等高级语言
- 产生了软件工程新技术
  - ✓ 提出了**瀑布软件开发过程模型，结构化软件开发方法学**
  - ✓ **形式化方法**（Formal Method）的研究非常活跃
- 研制了一些支持结构化软件开发方法学、形式化方法的CASE工具和环境
- 开始采用定量方法来指导软件开发、管理和质量保证

20世纪80年代 （1980 – 1990）

- 计算机软件的应用领域和范围不断扩大，软件数量、系统规模和复杂性不断增长
- 产生了**面向对象程序设计技术**
  - ✓如Smalltalk、C++等
- 提出了**SW-CMM**，即**软件能力成熟度模型**
- **软件重用**被视为是解决软件危机的一条现实可行途径
- **CASE工具和环境**的研制和使用成为热点
- **软件工程标准化**工作非常活跃，成果丰硕

20世纪90年代 （1990 – 2000）

- 局域计算环境开始流行，互联网应用开始出现
- OOP技术趋于成熟，面向对象分析和设计方法学的研究非常活跃，逐步形成系统化的面向对象软件工程
  - ✓制定面向对象建模语言UML，产生了统一软件开发过程RUP
- 软构件技术得到了快速发展，萌生了软件体系结构和软件设计模式的研究与实践
- 开源软件及技术开始出现
- 人机交互技术取得长足进步

21世纪前十年 （2000 – 2010）

- 互联网技术日趋成熟，信息技术快速发展，软件数量不断增长，越来越多的软件部署在互联网上运行并提供服务
- 产生了网构软件技术、自适应软件工程、可信软件技术、面向主体软件工程等
- 群体化软件开发技术在开源软件开发实践中广泛应用
- 面向服务软件工程的研究与实践
- 敏捷开发方法的在软件开发中的应用
- 模型驱动软件开发技术的研究与应用
- 软件可信技术的研究与实践非常活跃

近十年 （2011 – 至今）

- 移动互联网得到了快速发展，应用软件需求激增，信息系统的人机物融合趋势日趋突出
- 人类正进入到软件定义一切（Software-Defined X, SDX）的时代，从而对软件开发和运维提出了严峻的挑战
- 越来越多的企业和个人参与开源软件实践，涌现形成了规模极为庞大的开源软件生态
- DevOps方法在软件产业界和软件开发实践中的广泛应用
- 智能化软件开发技术研究活跃
- 大规模复杂系统软件智能运维、故障检测、根因分析等
- 大模型时代软件工程新方法

生成式软件开发

- **以大语言模型为核心引擎，通过自然语言交互（提示词）、工作流编排和能力集成，快速构建智能化软件的新型范式**
- **技术基础**
  - 大模型及代码大模型基座
  - 提示(Prompt)工程、检索增强生成(RAG)、思维链(CoT)等
  - 多智能体(Agent)技术以及相关的上下文工程、记忆工程等
- **与传统软件开发的区别**
  - ✓ 传统软件开发：重点是需求分析、软件设计和编码实现，需要较强的计算机和软件工程方面的专业基础
  - ✓ 生成式软件开发：重点是问题定义与拆解、交互流程设计以及现有能力组合，适合普通用户构建个性化的软件工具或应用

氛围编程(Vibe Coding)：像聊天一样”写”代码

- 氛围编程是2025年由OpenAI联合创始人安德烈-卡帕西Andrej Karpathy提出的新型编程范式，其核心理念是“忘记代码的存在，专注于想法的实现”
- 它依赖大模型及Agent等AI技术以及智能化IDE(集成开发环境)，开发者通过自然语言描述需求，AI直接生成代码逻辑与架构，开发者仅需监督流程和测试功能

主要优势

- **对话式开发**：像跟朋友聊天一样提需求
- **超快原型**：几分钟做出可运行的网页/小工具
- **零门槛**：无需先学习编程语言、开发框架等

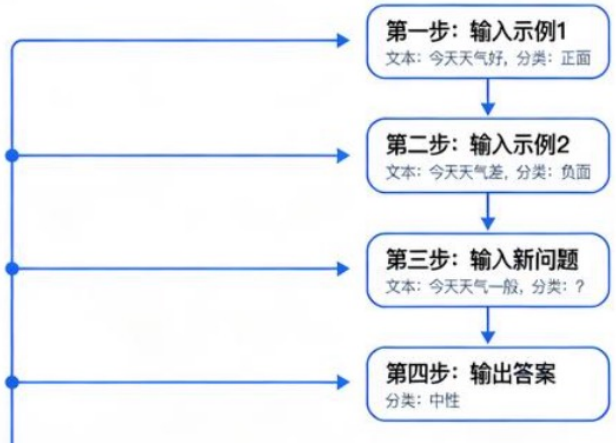
潜在问题

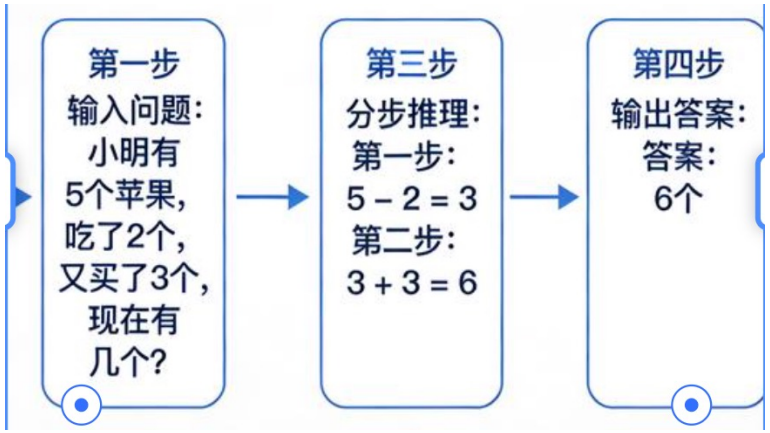
- **精确性不足**：对用户需求实现的精确性不足
- **技术债与安全隐患**：暗含技术债与安全隐患等长期维护可能遇到的问题
- **各种“脑补”的细节**：代码实现中暗含各种“脑补”的假设以及画蛇添足的功能
- **持续迭代与修改的难题**：持续修复问题和迭代完善可能越改越糟糕

⚠ 氛围编程**适合“从0到1”的创意激发**，而“从1到100”的打磨优化，仍然需要人的**逻辑思考和工程素养**

提示词工程 (Prompt Engineering)

- **提示工程:** 通过设计和优化提供给大语言模型的**提示词(Prompt)**以**引导模型生成更准确、更符合需求**的输出的一种技术方法
- **类比**: 如何在向他人求教或布置任务的时候清晰表达自己的需求以及需要提供给他人的额外提示 (例如，注意事项、举例说明等)?
- **方式:**
  - ✓ **示例增强**（Few-shot）：通过少量示例引导模型学习格式
  - ✓ **思维链增强**（CoT）：引导模型分步推理，提升复杂任务准确性

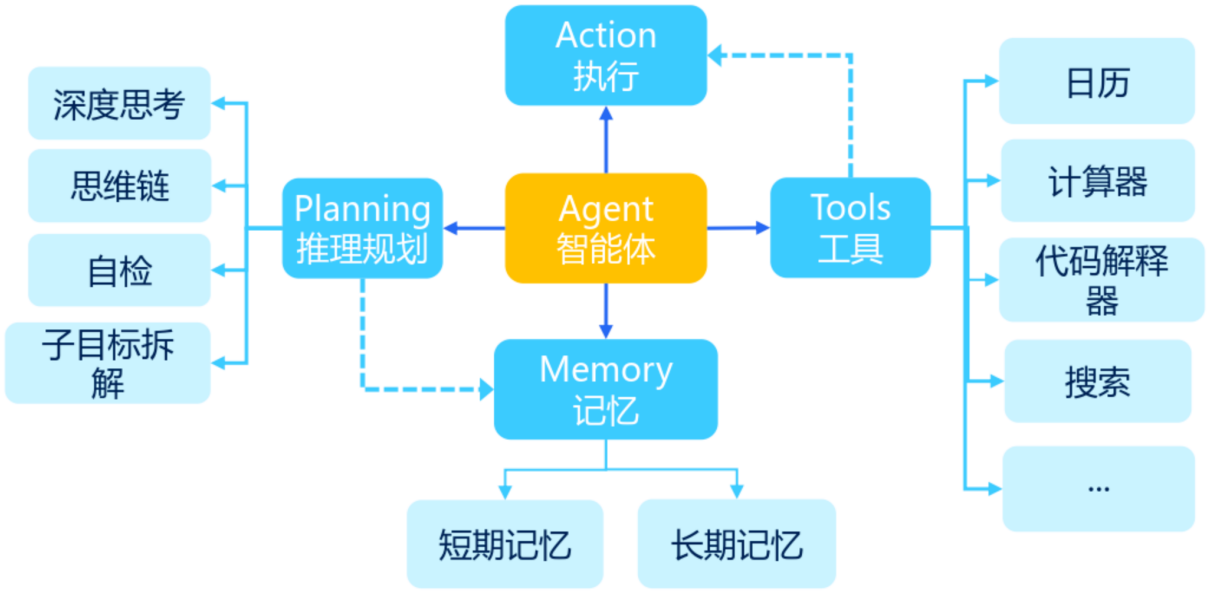




智能体

- **Agent是以大模型为决策中枢(大脑)，能自主完成任务的智能实体**
- **能根据用户最初下达的指令，长时间+独立自主地工作**

- ✓ **感知(Perception)**：理解用户的自然语言指令
- ✓ **规划与推理（Planning&Reasoning）**：借助模型推理能力，将任务拆解为查询往返机票、筛选酒店、经典规划等子任务
- ✓ **记忆（Memory):** 记住用户偏好，例如靠窗、快捷酒店
- ✓ **工具(Tool):**调用外部工具，例如机票预定 API、地图服务等，来执行子任务
- ✓ **反馈与迭代（Feedback&Iteration）**：将规划好的草案反馈用户，根据修改意见进行调整，完成最后动作



软件工程的知识领域

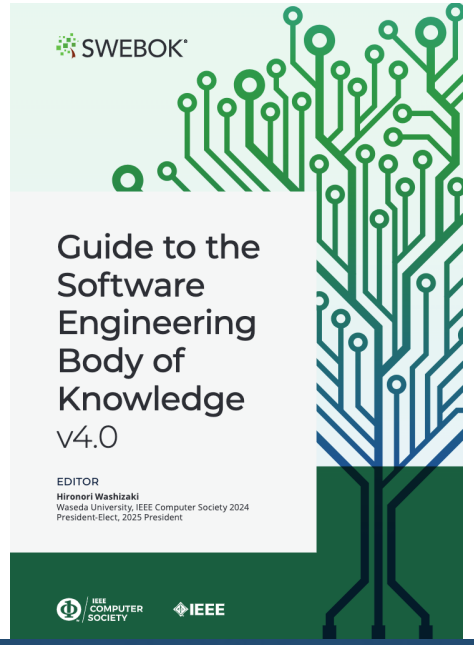
❑ IEEE SWEBOK V3.0 (SoftWare Engineering Body of Knowledge)

- ✓ 知识域(Knowledge Area: KA)
- ✓ Reviewed by approximately 150 reviewers in 33 countries

- ❑ 软件需求
- ❑ 软件设计
- ❑ 软件构造
- ❑ 软件测试
- ❑ 软件维护
- ❑ 软件配置管理
- ❑ 软件工程管理
- ❑ 软件工程过程
- ❑ 软件工程建模与方法
- ❑ 软件质量
- ❑ 软件工程专业实践
- ❑ 软件工程经济学
- ❑ 计算基础
- ❑ 数学基础
- ❑ 工程基础

❑ IEEE SWEBOK V4.0

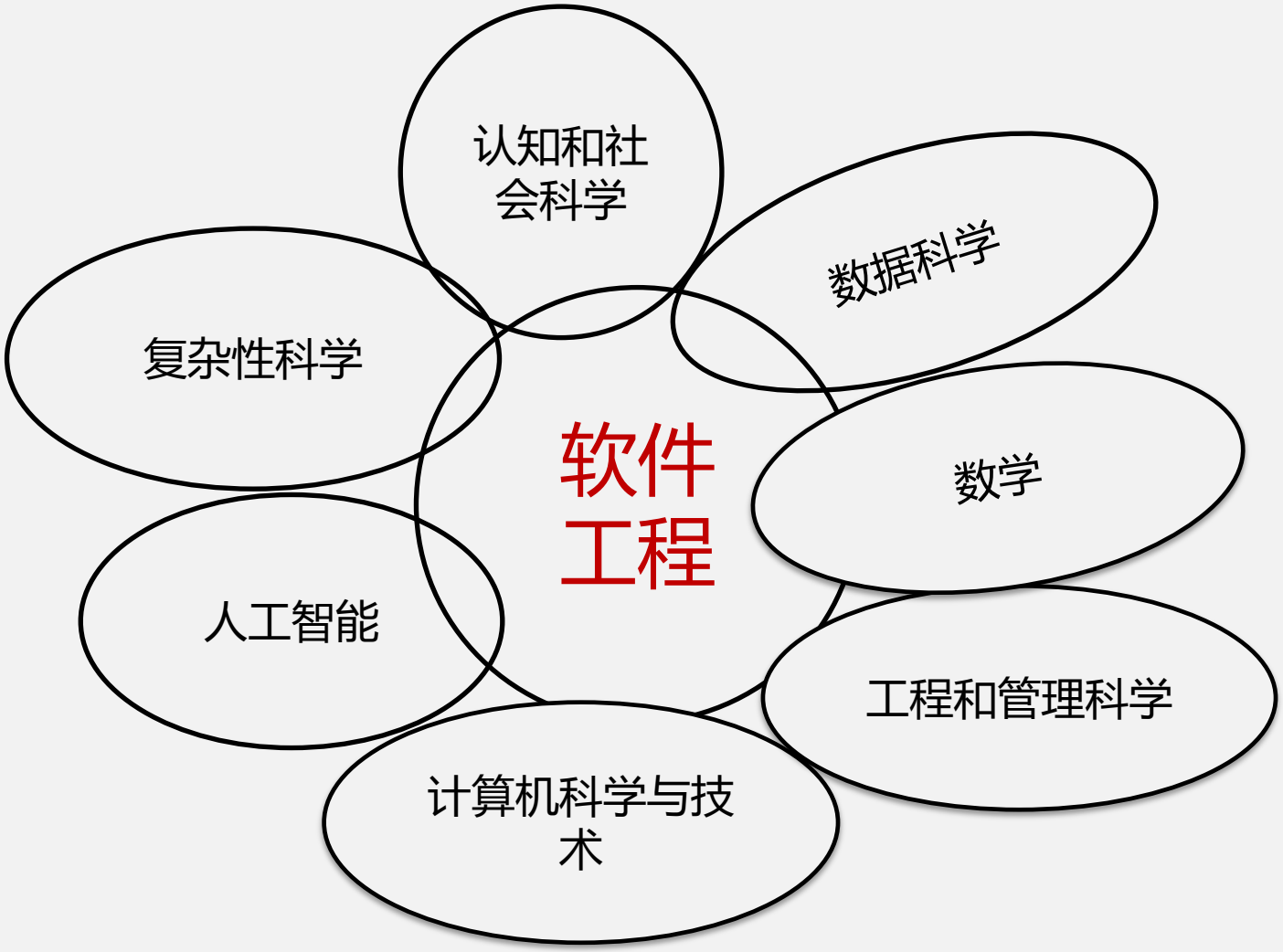
- ✓ 新增三个知识领域
  - ✓ 软件体系结构
  - ✓ 软件工程运维
  - ✓ 软件安全



V3 (2014)	V4 (2024)
Introduction	Introduction
1. Software Requirements	1. Software Requirements
	2. Software Architecture
2. Software Design	3. Software Design
3. Software Construction	4. Software Construction
4. Software Testing	5. Software Testing
	6. Software Engineering Operations
5. Software Maintenance	7. Software Maintenance
6. Software Configuration Management	8. Software Configuration Management
7. Software Engineering Management	9. Software Engineering Management
8. Software Engineering Process	10. Software Engineering Process
9. Software Engineering Models and Methods	11. Software Engineering Models and Methods
10. Software Quality	12. Software Quality
	13. Software Security
11. Software Engineering Professional Practice	14. Software Engineering Professional Practice
12. Software Engineering Economics	15. Software Engineering Economics
13. Computing Foundations	16. Computing Foundations
14. Mathematical Foundations	17. Mathematical Foundations
15. Engineering Foundations	18. Engineering Foundations
Appendix A. Knowledge Area Specifications	Appendix A. Knowledge Area Specifications
Appendix B. Standards	Appendix B. Standards
Appendix C. Consolidated Reference List	Appendix C. Consolidated Reference List

https://ieeecs-media.computer.org/media/education/swebok/swebok-v4.pdf

软件工程的多学科交叉



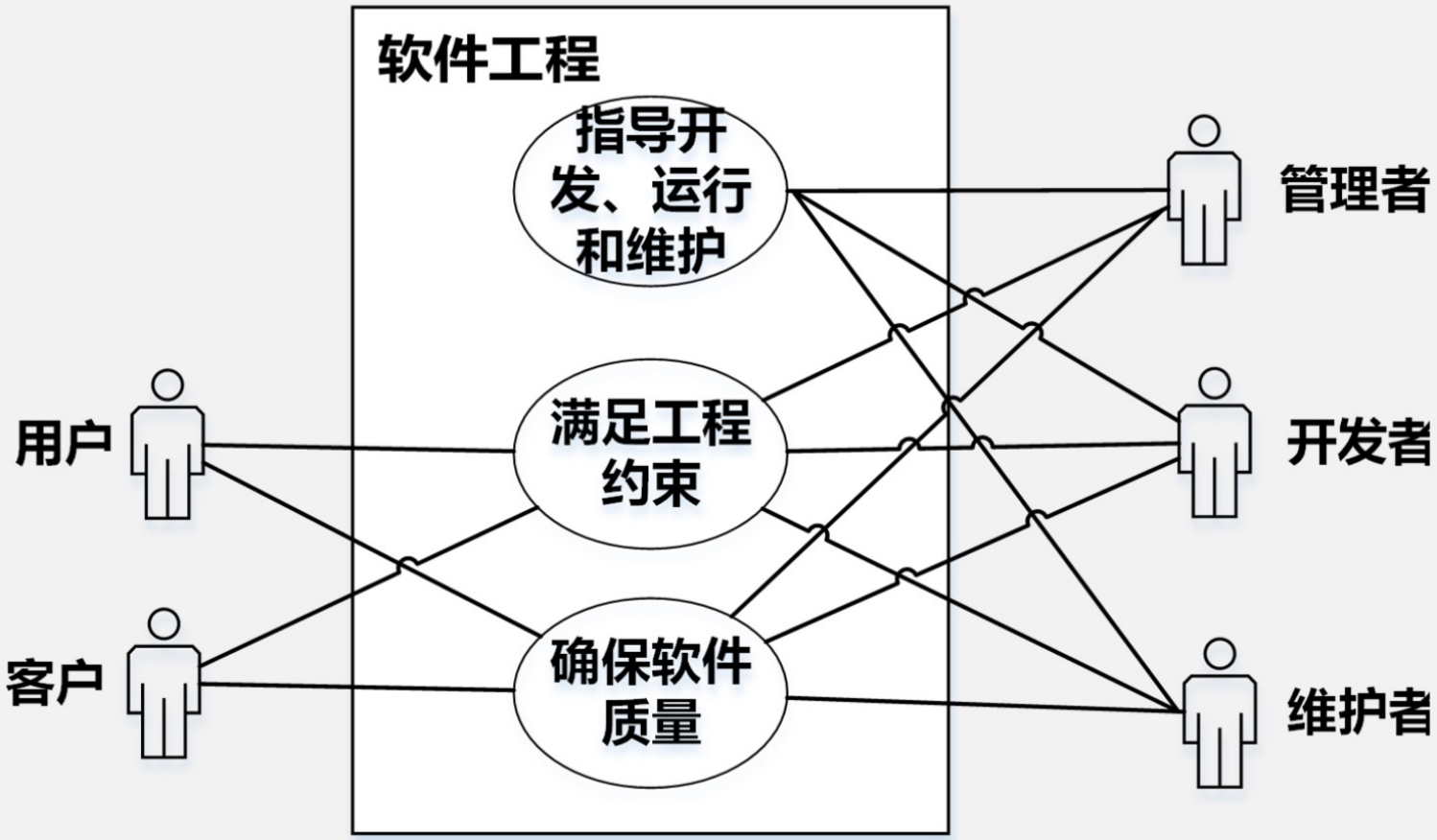
- ✓ 认识大型复杂软件系统
- ✓ 揭示和解释内在的机理
- ✓ 指导方法的研究与实践
- ✓ ……

主要内容

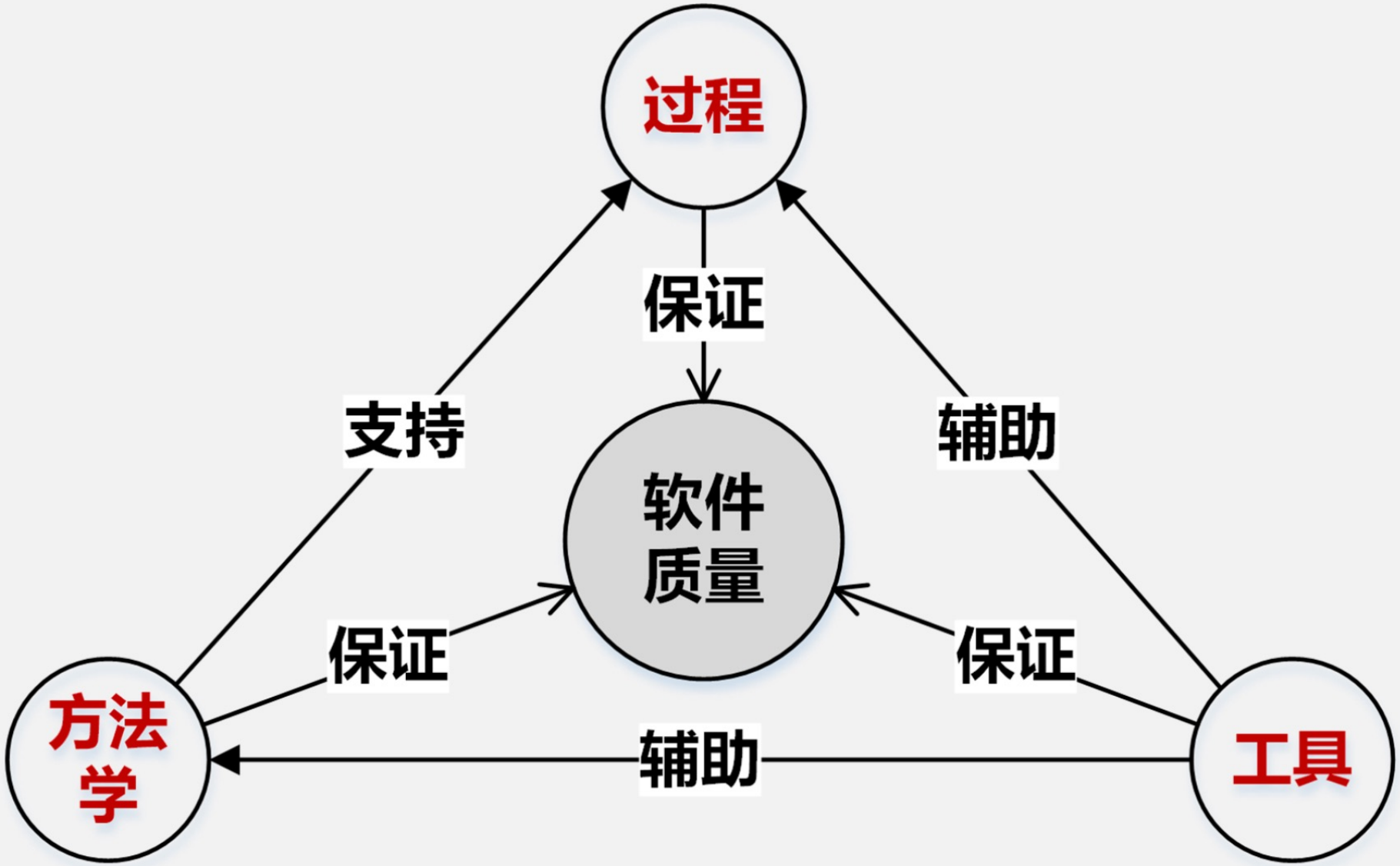
- 软件工程定义
- 软件工程发展历史
- 软件工程目标和三要素
- 软件工程的基本原则与规范

软件工程的目标

- 在成本、进度等约束下，指导软件开发和运维，开发出满足用户要求的足够好软件



软件工程的三要素



过程(Process)

- 从**管理**的视角，回答软件开发、运行和维护需要开展**哪些工作、按照什么样的步骤和次序**来开展工作
- 对软件开发过程所涉及的**人、制品、质量、成本、计划等**进行**有效和可量化**的管理
- 典型成果
  - ✓**过程模型**，如瀑布模型、增量模型、原型模型、迭代模型、螺旋模型等等
  - ✓**方法**，如敏捷开发方法、群体化开发方法、DevOps方法
  - ✓**管理**，如配置管理、质量管理、团队组织等

方法学(Methodology)

- 从**技术**的视角，回答软件开发、运行和维护**如何做**的问题
- 为软件开发过程中的各项开发和维护活动提供**系统性、规范性**的技术支持
  - ✓如何**理解和认识软件模型**是什么
  - ✓如何用**不同抽象层次**的模型来**描述软件制品**
  - ✓采用什么样的**建模语言**来描述**软件模型**等等
- **典型成果**
  - ✓结构化软件开发方法学
  - ✓面向对象软件开发方法学
  - ✓基于构件的软件开发方法学
  - ✓面向服务的软件开发方法学

工具(Tool)

- **从工具辅助的视角，主要回答如何借助工具来辅助软件开发、运行和维护的问题**
- **帮助软件开发人员更为高效地运用软件开发方法学来完成软件开发过程中的各项工作，提高软件开发效率和质量，加快软件交付进度。**
  - ✓如需求分析、软件设计、编码实现、软件测试、部署运行、软件维护、项目管理、质量保证等，简化软件开发任务，
- **典型成果**
  - ✓SonarQube、Eclipse、Visual Studio等

主要内容

- 软件工程定义
- 软件工程发展历史
- 软件工程目标和三要素
- 软件工程的基本原则与规范

思考：编写程序用到的软件工程原则

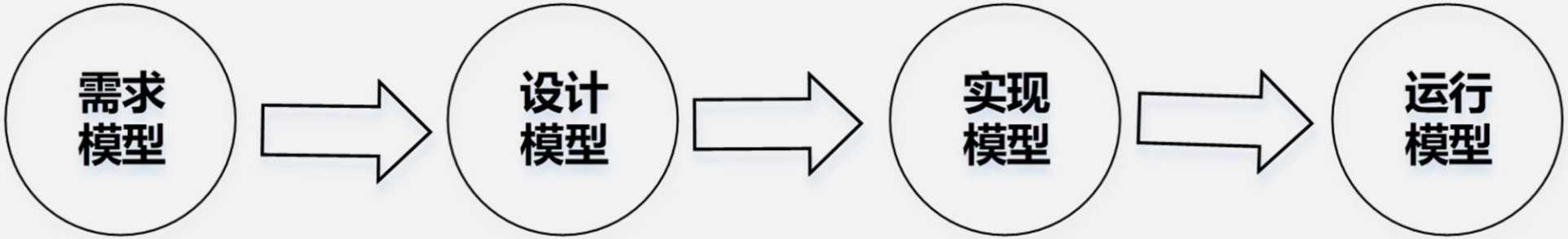
软件工程基本原则

抽象和建模、模块化、软件重用、信息隐藏、关注点分离、分而治之、双向追踪、工具辅助

软件工程原则（1）

❑抽象和建模

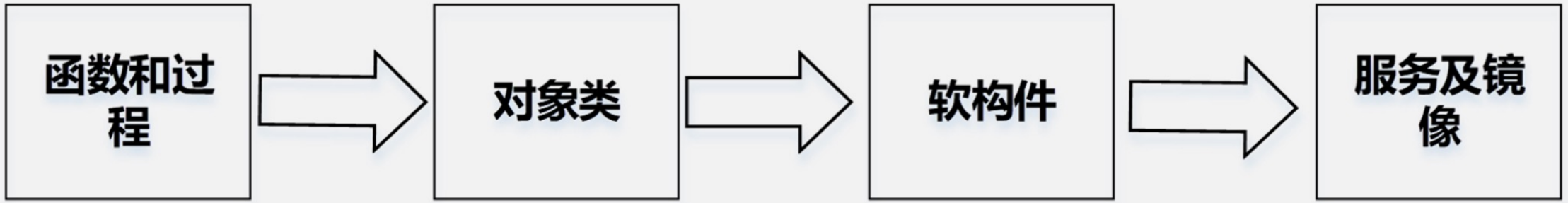
- ✓**抽象**：将与相关开发活动**所关注的要素**提取出来，不关心的要素扔掉，形成与该开发活动相关的**软件要素**
- ✓**建模**：基于特定的抽象，借助于**建模语言**（如数据流图、UML等），建立起基于这些抽象的**软件模型**，进而促进对软件系统的准确理解



软件工程原则（2）

❑模块化

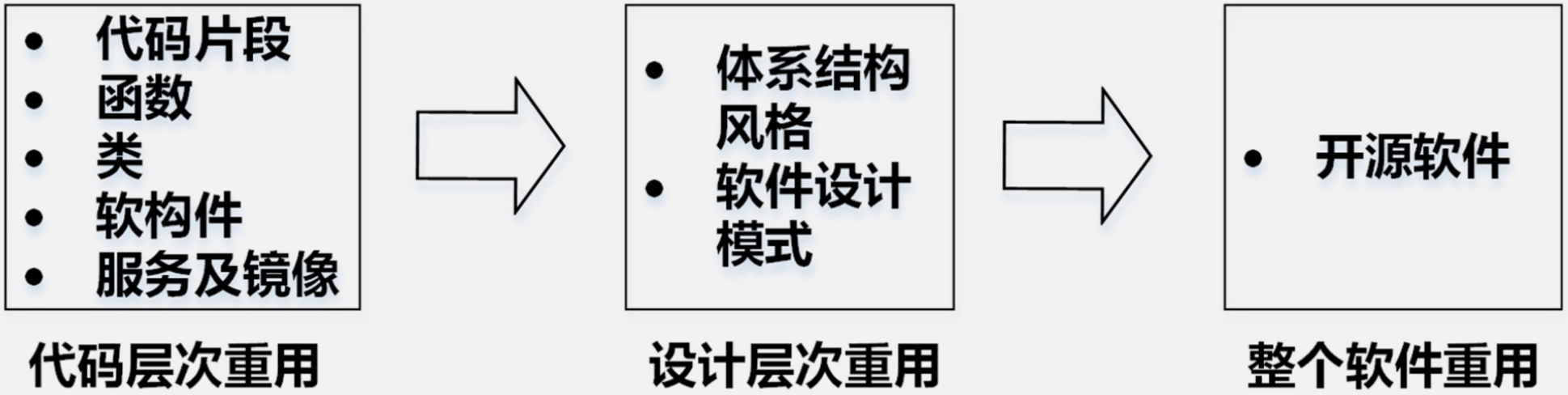
- ✓将软件系统的**功能分解**和实现为若干个模块，每个模块具有**独立的功能**，模块之间通过接口进行调用和访问。
- ✓**模块内部高内聚，模块间松耦合**



软件工程原则（3）

❑软件重用

- ✓在软件开发过程中尽可能**利用已有的软件资源和资产**（如函数库、类库、构件库、开源软件、代码片段等）来实现软件系统
- ✓努力开发出**可被再次重用**的软件资源（如函数、类、构件等）
- ✓有助于提高软件开发效率，降低软件开发成本，满足开发工程约束，得到高质量的软件产品



软件工程原则（4）

❑信息隐藏

- ✓模块内部信息（如内部的语句、变量等）对外**不可见或不可访问**，模块间仅仅交换那些为完成系统功能所必需交换的信息（如接口）
- ✓模块设计时**只对外提供可见的接口**，不提供内部实现细节。信息隐藏原则可提升模块的独立性，减少错误向外传播，支持模块的并行开发

软件工程原则（5）

❑关注点分离

- ✓在软件开发过程中，将若干性质不同的**关注点分离**开来，以便在不同的开发活动中针对不同的关注点，随后将这些关注点的开发结果整合起来，形成关于软件系统的完整视图
- ✓软件系统具有**多面性的特点**，既有**结构特征**，如软件的体系结构，也有**行为特征**，如软件要完成的动作及输出的结果；既有高层的**需求模型**，描述了软件需要做什么，也有低层的**实现模型**，描述了这些需求是如何实现的
- ✓使得开发者在**每一项开发活动中聚焦于某个关注点**，有助于简化开发任务；同时通过**整合多个不同视点**的开发结果，可获得关于软件系统的更为清晰、系统和深入地认识

软件工程原则（6）

❑分而治之

- ✓在软件开发和维护过程中，软件开发人员可**对复杂软件系统进行分解**，形成一组子系统
- ✓如果子系统仍很复杂，还可以继续进行分解，及至通过分解所得到的子系统易于处理；然后通过**整合**子系统的问题解决得到整个系统的问题解决
- ✓有助于简化复杂软件系统的开发，降低软件开发复杂性，从而提高软件开发效率，确保复杂软件系统的质量。

软件工程原则（7）

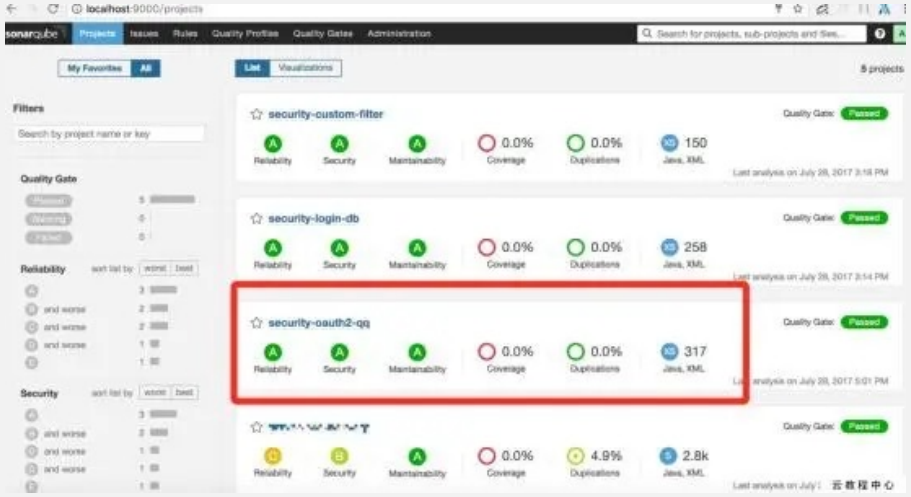
❑双向追踪原则

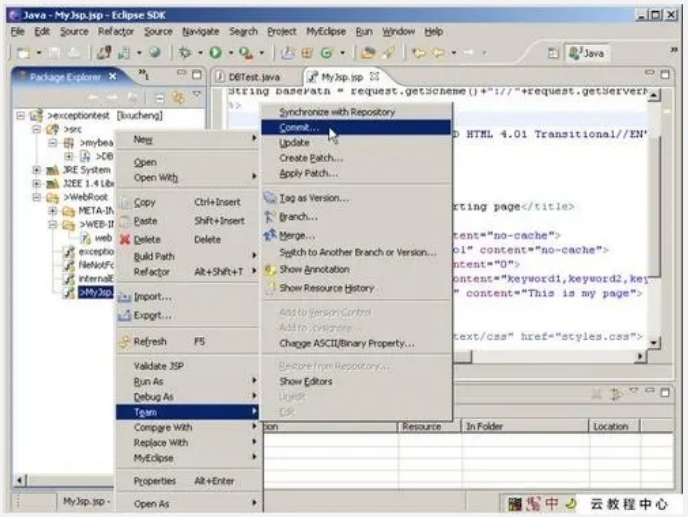
- ✓当某个软件制品**发生变化**时，一方面要追踪这种**变化会对那些软件制品产生影响**，进而指导相关的开发和维护工作，此为正向追踪；另一方面要**追踪产生这种变化的来源**，或者说是什么因素导致了该软件制品的变化，明确软件制品发生变化的原因及其合理性，此为反向追踪。
- ✓有助于确保软件制品间的一致性，发现无意义的变化，并基于变化指导软件的开发和维护，确保软件质量。

软件工程原则（8）

❑工具辅助

- ✓利用软件工具来辅助软件开发和维护工作是一项行之有效的方法
- ✓尽可能地借助计算机工具来辅助软件开发和维护，以降低开发者和维护者的工作负担，提高软件开发和维护效率，提升软件开发及软件制品的质量





工欲善其事必先利其器

软件工程的基本观点

- 在选择开发工具上，软件工程的基本观点是：工具**不是越新越好**，而是你越熟悉越好；
- 在风险评估上，软件工程的基本观点是：任何项目组都**存在技术风险和技能风险**；
- 在项目策划上，软件工程的基本观点是：**需求分析在前，项目方案策划在后**；
- 在项目开发进度上，软件工程的基本观点是：**宁愿慢些**，也要保证软件的质量；

软件工程的基本观点

- 在项目质量上，软件工程的基本观点是：质量蕴涵于开发过程之中，而不是过程之后；
- 在项目人员上，软件工程的基本观点是：项目组在中途靠增加开发人员来提高开发速度是无效的 -（人月神话）
- 在项目经理的职责上，软件工程的基本观点是： 80% 的时间是写文档， 20% 的时间是内部勾通；
- ……

小结

- **软件工程产生的背景和目的**
  - ✓软件危机，持续存在，关注点不同
- **软件工程的本质**
  - ✓软件视为产品，软件开发视为工程、创作和生产相结合的过程
  - ✓三要素：过程、方法学和工具
  - ✓软件工程的基本原则
- **软件工程的发展**
  - ✓不同阶段和时期，不同的思想和技术
- **软件工程教育**
  - ✓软件工程知识体系及课程特点

北京航空航天大學
BEIHANG UNIVERSITY

Thank you

北京航空航天大学